

Version 11

Programming Guide

SecurOS REST API Programming Guide (PG - EN, build 181 on 27.12.2021).

© Copyright Intelligent Security Systems, 2021.

Printed in US.

Intelligent Security Systems reserves the right to make changes to both this Manual and to the products it describes. System specifications are subject to change without notice. Nothing contained within this Manual is intended as any offer, warranty, promise or contractual condition, and must not be taken as such.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system or translated into any human or computer language in any form by any means without the express written permission of the copyright holder. Unauthorized copying of this publication may not only infringe copyright but also reduce the ability of Intelligent Security Systems to provide accurate and up-to-date information to both users and operators.

Contents

1 Preface 6
1.1 Scope 6
1.2 Target Audience 6
1.3 Using This Manual 6
1.4 Getting Technical Support 6
1.5 SecurOS Editions Naming Convention 8
1.6 Design Convention 8
1.7 Design Elements 9
2 General Description 10
3 Preparations for Use 11
4 Authorization In SecurOS 13
4.1 Authorization When Using HTTP/HTTPS 13
4.2 Authorization When Using WebSocket 13
5 Accessing Resources 16
6 Working With Cameras 17
6.1 Get Camera Settings And State 17
6.2 Get State of Specified Camera 21
6.3 Get Camera Connection Settings 23
6.4 Change Connection Settings of Specified Camera/Rename Camera 26
6.5 Delete Camera 29
6.6 Working With PTZ Camera 30
6.6.1 Rotate/Scale Camera 30
6.6.2 Rotate/Scale Camera Using Absolute Coordinates 31
6.6.3 Get PTZ Camera State 32
6.6.4 Get Tour and Preset List 33
6.6.5 Go To Preset 34
6.6.6 Start Patrol 34
6.6.7 Stop Patrol 35
6.7 Camera Recording Control 36
6.7.1 Start Record 36
6.7.2 Start Record and Add Pre-recorded Video from Pre-recording Buffer 36
6.7.3 Stop Record 37
6.8 Working with Cameras of RTSP Server 38
6.8.1 Get Information about RTSP Streams of Camera 38
6.8.2 Get Information about Live Video RTSP Stream of Camera 39
6.8.3 Get Information about Archive Video RTSP Stream of Camera 40
6.9 Working with Washer and Wiper 41
6.9.1 Start Wiper 41
6.9.2 Start Washing 41

7 Video Archive Export 43
7.1 Create Archive Export Task 43
7.2 Get Information about All Current Archive Export Tasks 44
7.3 Get Information about Specified Archive Export Task 45
8 Frame Export 47
8.1 Get Frame of Live Video from Specified Camera 47
8.2 Get Frame of Archive Video from Specified Camera 48
8.3 Get Frame of Specified Resolution from Specified Camera 48
9 Working with User Accounts 50
9.1 Get Information about All Departments 50
9.2 Get Information about Specified Department 51
9.3 Get Data of All User Accounts 51
9.4 Get Data of Specified User Account 52
9.5 Create User Account 53
9.6 Change User Account Settings 54
9.7 Delete User Account 55
10 Working with Events 57
10.1 Event Filtering 57
10.2 Get Event List for Specified Camera 57
10.3 Send Custom Event to SecurOS 58
11 Working with Map 60
11.1 Get Settings of All Maps 60
11.2 Get Settings of Specified Map 62
12 Working with SecurOS Auto 64
12.1 Get List of Logic Modules 64
12.2 Get Settings of All Recognizers 65
12.3 Get Settings of Specific Recognizer 66
12.4 Get List of Passages for Time Interval 67
12.5 Get Snapshot of Specific Passage 69
12.6 Managing Watchlists 70
12.6.1 Get List of Watchlists 70
12.6.2 Add License Plate Number to Watchlist or Update Its Comment 71
12.6.3 Remove License Plate Number from Watchlist 72
12.6.4 Check if License Plate Number Belongs to Watchlist 73
12.7 Subscribe to Notifications about New Passages 74
13 Working with FaceX 75
13.1 Get FaceX Server Settings 75
14 Working with ACS 77

14.1 Get List of ACS Objects of Specific Type . . . 77
15 Working with Macros 79
15.1 Get Settings of All Macros . . . 79
15.2 Get Settings of Specified Macro . . . 80
15.3 Run Macro . . . 80
16 Working with Computers 82
16.1 Get Settings of All Computers . . . 82
16.2 Get Settings of Specified Computer . . . 84
17 Get License Information 87
18 Working with Subscriptions 89
18.1 Working via HTTP . . . 89
18.1.1 Subscribe to event (HTTP) . . . 89
18.1.2 Get Information about All Active Subscriptions (HTTP) . . . 90
18.1.3 Get Information about Specified Subscription (HTTP) . . . 92
18.1.4 Update Subscription (HTTP) . . . 92
18.1.5 Delete Subscription (HTTP) . . . 93
18.2 Working via WebSocket . . . 94
18.2.1 Monitoring SecurOS server connection state . . . 95
18.2.2 Common Request . . . 95
18.2.3 Subscribe to Event (WebSocket) . . . 96
18.2.4 Subscribe to State Change (WebSocket) . . . 97
18.2.5 Get Information about All Active Subscriptions (WebSocket) . . . 98
18.2.6 Delete All Subscriptions (WebSocket) . . . 101
18.2.7 Delete Subscription (WebSocket) . . . 101
18.2.8 Update Subscription (WebSocket) . . . 102
18.2.9 WebSocket Server Messages . . . 102
18.2.9.1 Object State Change Message . . . 102
18.2.9.2 Event Message . . . 103
18.2.9.3 Object Deletion Message . . . 104
18.2.9.4 Error Message . . . 105
19 HTTP Status Codes 106
20 Technical Support Information 107

1 Preface

This section contains general information about this document, the means of its design and use, as well as how to get additional technical support for the product.

1.1 Scope

This guide describes SecurOS REST API resources used to obtain system information, configure the system, and manage system objects.

1.2 Target Audience

This guide is intended for developers who integrating external systems with SecurOS.

1.3 Using This Manual

This document is organized in such a way that the user can use both its printed and electronic versions. In the latter case one can use Adobe Reader's Bookmarks feature as well as cross-reference hyperlinks to navigate through content. In several topics this manual refers to other SecurOS manuals (for example, **SecurOS Quick User Guide**). One can find these manuals as separate files on the SecurOS installation CD or download them from our website (**www.issivs.com**).

To get online help (Microsoft HTML Help) just press the **F1** key when running SecurOS in administrative mode. You can get context help for a given object by pressing the **F1** key when its settings window is open.

1.4 Getting Technical Support

If you have any questions after reading this manual, please address them to your system administrator or supervisor.

For any further information you can contact the Intelligent Security Systems Technical Support Team:

Note. To get a quick response to a request use the Technical Support Portal, which www address is listed below.

- **in USA:**

  phone: +1 732 855 1111 (Monday to Friday, 8:30am - 6pm EST);

  e-mail: **support@issivs.com**

  www: **https://support.issivs.com**

- **in Russia:**
  phone: +7 (495) 645 21 21 (Monday to Thursday, 9am - 6pm MST; Friday 9am - 5pm MST);
  www: **https://help.iss.ru**

Note. See the **https://help.iss.ru/user/manual** for the Portal User Guide.

- **in Brazil:**
  phone: +55 11 2262 2894 (Monday to Friday, 9am - 6pm BRT);
  e-mail: **suporte@issivs.com**
  www: **https://support.issivs.com**
- **in Mexico:**
  phone: +52 1 551330 0181 (Monday to Friday, 9am - 6pm CDT);
  e-mail: **supportlatam@issivs.com**
  www: **https://support.issivs.com**
- **in Colombia/Ecuador:**
  phone: +57 300 442 2808 (Monday to Friday, 9am - 6pm COT/ECT);
  e-mail: **supportlatam@issivs.com**
  www: **https://support.issivs.com**
- **in Chile:**
  phone: +56 9 6573 2993 (Monday to Friday, 9am - 6pm CLT);
  e-mail: **supportlatam@issivs.com**
  www: **https://support.issivs.com**
- **in Ukraine:**
  phone: +380 (44) 299 08 10 (Monday to Friday, 9am - 6pm EET);
  e-mail: **supportua@issivs.com**
  www: **https://support.issivs.com**
- **in Peru/Bolivia:**
  phone: +51 997 111 678 (Monday to Friday, 9am - 6pm PET/BOT);
  e-mail: **supportlatam@issivs.com**
  www: **https://support.issivs.com**
- **in Argentina:**
  phone: +54 91152528779 (Monday to Friday, 9am - 6pm ART);
  e-mail: **supportlatam@issivs.com**
  www: **https://support.issivs.com**

To solve problems faster, we recommend preparing the service information described in the **Technical Support Information** Section before addressing the Technical Support Team.

1.5 SecurOS Editions Naming Convention

This document represents a common manual for several editions of the "SecurOS integrated video management platform" that differ in functional capabilities:

- *SecurOS Monitoring & Control Center PA*;
- *SecurOS Monitoring & Control Center*;
- *SecurOS Datacenter*;
- *SecurOS Enterprise*;
- *SecurOS Premium*;
- *SecurOS Professional*;
- *SecurOS Xpress*;
- *SecurOS Lite*.

For product designation regardless of its edition the *SecurOS* general term is used in the framework of the given document.

For designation of the *SecurOS Monitoring & Control Center PA* and the *SecurOS Monitoring & Control Center* editions the *SecurOS Monitoring & Control Center* general term is used unless otherwise specified.

Sections that describe the functionality available for some editions are marked by a special footnote as in the example below:

> The functionality is available in the following editions: *SecurOS Monitoring & Control Center, SecurOS Datacenter, SecurOS Enterprise, SecurOS Premium, SecurOS Professional, SecurOS Xpress, SecurOS Lite.*

1.6 Design Convention

For representation of various terms and titles the following fonts and formatting tools are used in this document.

Font	Description
bold type	Used in writing workstation names, utilities or screens, windows and dialog boxes as well as the names of their elements (GUI elements).
italic type	Used to mark out the SecurOS objects.
bold italic type	Used to mark out the elements of homogeneous lists.
`monospace`	Used to mark out macro text and programming code, file names and their paths. Also it is used to specify the necessary options, to mark out values specified by the user from the keyboard (manually).
green	Used to mark out the cross-references within the document and links to the external available ones.

1.7 Design Elements

Warning! Serves to alert the user to information which is necessary for the correct perception of the text set out below. Typically, this information has a warning character.

Note. Note text in topic body.

Additional Information

Used to display additional information. These type of elements contain, for example, the description of options for executing a task or reference to additional literature.

2 General Description

SecurOS REST API is a RESTful web-service that allows you to receive information about the system, configure it or manage system objects. All methods are implemented as web resources, which can be accessed using the HTTP / HTTPS or WebSocket protocols.

HTTP/HTTPS communication is organized as request-and-response via REST requests. REST requests allow to control some SecurOS components and observe their settings and status. The list of provided requests is presented in the table of contents. Each section describes one REST request. Requests are grouped by the components they addressed to.

WebSocket protocol is used to **subscribe** to SecurOS events and state changes. It is also used to receive notification about new events. Those events may occur as a result of REST request and due to other reasons.

Note. WebSocket Extensions and WebSocket Subprotocols are not supported.

---

3 Preparations for Use

To use the REST API methods to interact with SecurOS perform the following preliminary settings:

- **Configuring SecurOS**.
- **Configuring HTTPS on the Client Side**.
- **Configuring Resource to Export Archive**.

Note. The last two procedures are optional and are necessary if you intend to use the HTTPS protocol and archive export feature.

Configuring SecurOS

To enable REST API capabilities it is required to create *REST API* object in the Object Tree under the *Computer* to which you are going to connect and specify the HTTP/HTTPS and WebSocket ports. For the details see **SecurOS Administration Guide**.

Configuring HTTPS on the Client Side

The following solution is based on a guide: https://blogs.msdn.microsoft.com/jpsanders/2009/09/29/how-to-walkthrough-using-httplistener-or-http-server-unmanaged-code-c-as-an-ssl-simple-server/.

To allow using HTTPS it is necessary to make a listener for a certain port with a certain SSL certificate.

To provide this it is necessary to do the following:

1. Get the thumbprint of the SSL certificate.
2. Link the certificate to a port which is used for REST API.
3. Create a GUID for the application (required to link certificate).

1. Depending on the source of the certificate its thumbprint can be obtained one way or another. If you are running Microsoft IIS then you can do the following:
   1.1. Open the Microsoft Management Console (MMC) and add the **Certificates (Local Computer)** snap-in.
   1.2. In the **Console Root** left pane click the **Certificates (Local Computer)**.
   1.3. Click the **Personal** folder to expand it.
   1.4. Click the **Certificates** folder to expand it.
   1.5. In the list of the certificates note the **Intended Purposes** heading. Find a certificate that lists **Client Authentication** as an intended purpose.
   1.6. Double-click the certificate.
   1.7. In the **Certificate** dialog box click the **Details** tab.
   1.8. Scroll through the list of fields and click the **Thumbprint** field.

Copy the hexadecimal characters from the box. If this thumbprint is used in code for the **X509FindType**, remove the spaces between the hexadecimal numbers. For example, the thumbprint "a9 09 50 2d d8 2a e4 14 33 e6 f8 38 86 b0 0d 42 77 a3 2a 7b" should be specified as "a909502dd82ae41433e6f83886b00d4277a32a7b" in code.

Also if necessary see https://msdn.microsoft.com/en-us/library/ms734695(v=vs.110).aspx.

2. Certificate port should be the same as the port specified in the *REST API* object settings (by default is `8888`).
3. To create a GUID it is possible to use any tool, for example, `https://www.guidgenerator.com/online-guid-generator.aspx`.

After finishing all preparations it's necessary to run Windows command line tool (**cmd**) as Administrator and execute the following string:

```
netsh http add sslcert ipport=0.0.0.0:8888
certhash=9526b167879d2050243b67842e7787fef83f9359
appid={FCB0C645-E745-490F-9E7D-2171E03DC9B0}
```
, where:

- **ipport —** the `0.0.0.0` must be set as IP address. Value, specified in the SecurOS *REST API* object settings (**HTTP port**) must be set as port number;
- **certhash —** the previously saved thumbprint (without spaces);
- **appid —** previously prepared application GUID.

To check if everything is done right run the following command:

```
netsh http show sslcert
```

In the end of received list there should be newly added certificate. And now it is possible to access SecurOS via REST API using HTTPS protocol.

Configuring Resource to Export Archive

Files exported using REST API methods are saved in a directory specified in the *Archive Converter* object settings. By default, a file prepared in this way is not available for download. To provide an access to this file one must do one of the following actions:

- Open network access to the folder where exported files are located;
- Deploy an HTTP server that will allow to download files;
- Deploy an FTP server which will grant access to the files.

In each of the three cases the link to the file will look different, but it will contain its own constant part. After choosing the file access method, this constant part should be set as the value of the **SharedExportPath** parameter.

For example, exported file must be accessed by the `http://contoso.my.server:8080/converted/<filename>` link. In this case, the constant part will be the `http://contoso.my.server:8080/converted/`. This string must be set as the **SharedExportPath** parameter value.

The parameter should be created in the `HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\ISS\SecurOS\Niss400\Core\RestApi` key of the Windows registry.

4 Authorization In SecurOS

To protect against unauthorized access, the SecurOS REST API integrates the Basic authorization.

Authorization details are described in the following sections:

- **Authorization When Using HTTP/HTTPS**.
- **Authorization When Using WebSocket**.

Note. Access to the REST API resources without authorization is impossible. For authorization use the credentials of any of the existing SecurOS *User Accounts* or Windows Active Directory. Authorization with the `superuser` credentials is impossible.

Warning! When executing requests, the rights of a successfully authorized user to the *REST API* object are not taken into account. At the same time, if this user does not have rights to some SecurOS objects or the current access level to the objects is insufficient, the corresponding requests will fail.

4.1 Authorization When Using HTTP/HTTPS

When using the HTTP/HTTPS protocol, each resource request must include an `Authorization` Header with SecurOS or Windows Active Directory user data for Basic authorization. An example with JScript authorization is presented below.

Example

```
var url = "http://172.16.1.120:8888/event?id=12"; // Port number from the REST API object settings
var login = "user_login";
var password = "user_password";

function HTTPRequest(request, url, login, password){
    var HTTP = new ActiveXObject("Msxml2.ServerXMLHTTP.6.0");
    HTTP.open("POST", url, false, login, password);
    HTTP.send();
    var response = new HTTP.responseText();
    return response;}
```

4.2 Authorization When Using WebSocket

When using WebSocket protocol authorization in SecurOS is performed in two stages:

1. **Getting Authorization Token for WebSocket Session**.
2. **Establishing an Authorized WebSocket Connection to the Server**.

Getting Authorization Token for WebSocket Session

To get authorization token send GET HTTP request to the SecurOS server where the *REST API* object is located. The request must contain data for Basic authorization (see **Authorization When Using HTTP/HTTPS**):

`http://<host>:<port>/api/v1/ws_auth`, where:

- `<host>` — DNS name or IP address of the server;
- `<port>` — port number specified in the *REST API* object settings for HTTP/HTTPS connection.

If the parameters are specified correctly the server will return the session authorization token:

```
{
  “data": { "token": "83cd26be3d014d2dbe7e9cdba455e47e" },
  “status": "success"
}
```

The token remains valid:

- within 10 minutes after it has been received;
- throughout the session after its authorization;
- within 10 minutes after closing the authorized connection.

Establishing an Authorized WebSocket Connection to the Server

After receiving the token a connection with the specified server is established. To authorize the connection pass the token received in the first message to the server (ws: // <host>: <websocket_port>):

```
{
 “type": "auth",
 “token": "83cd26be3d014d2dbe7e9cdba455e47e"
}
```

If authorization is passed the server responds with confirmation. The client is able to send any requests now:

```
{
  “data": { "token": "83cd26be3d014d2dbe7e9cdba455e47e" },
  "type": "auth success"
}
```

If an incorrect token is sent or it has expired (see above) the server will return the following error:

```
{
 “data":{
  “error": "INVALID_AUTH_TOKEN",
  “message": "Invalid authorization token"
 },
 “type": "error"
}
```

If the client sent the request to an unauthorized connection, the server will return the following error:

```
{
 “data": {
  “error": "UNAUTHORIZED",
  “message": "Unauthorized",
```

```
  “request_id": 1
 },
 “type": "error"
}
```

5 Accessing Resources

SecurOS REST API is a RESTful web-service that allows you to receive information about the system, configure it or manage system objects. All methods are implemented as a web resources. Each resource can be accessed by the URL, which is an HTTP link.

URL consists of two parts, in the `<Host:Port><URI>` format, where:

- `Host:Port` — SecurOS REST API server address and port number that the client is accessing (for example, `http://VIDEO_SERVER_I:8888` or `http://localhost:8888`). DNS name of the computer or IP address can be used;
- `URI` — resource identifier (for example, `/api/v1/cameras` or `/api/v1/persons`).

6 Working With Cameras

The following requests are provided for working with the *Cameras*:

- **Get Camera Settings And State**.
- **Get State of Specified Camera**.
- **Get Camera Connection Settings**.
- **Change Connection Settings of Specified Camera/Rename Camera**.
- **Delete Camera**.

See also:

- **Working With PTZ Camera**.
- **Camera Recording Control**.
- **Working with Cameras of RTSP Server**.
- **Working with Washer and Wiper**.

6.1 Get Camera Settings And State

Returns settings and state of *Cameras* of configuration.

Resources IDs

- `/api/v1/cameras` — to get settings and state of all *Cameras* of configuration as array.
- `/api/v1/cameras/<cam_id>` — to get settings and state of specific *Camera*.

Objects Access Rights

Camera — `View` () and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/cameras
```

Response Example

```
{
  "data": [
    {
      “id”:"3",
      "name": "Camera 3",
      "privilege": 5,
      "settings": {
        "PRESET.name.count": "0",
        "PRESET.preset.count": "0",
        "TOUR.name.count": "0",
        "TOUR.tour.count": "0",

       ... // Some parameters are excluded

        "send_forensic_data": "",
        "sharpness": "",
        "short_exposures": "",
        "telemetry_id": "native",
        "wiper": ""
      },
      "status": {
        "enabled": true,
        "focused": "true",
        "hold_ptz_SLAVE_ID": "",
        "hold_ptz_module_id": "",
        "hold_ptz_priority": "",
        "hold_ptz_user_id": "",
        "priority": "",
        "recorded": "0",
        "state": "DISARMED",
        "unblinded": "true",
        "valid": true
      },
      "type": "CAM"
    },
    {
      “id”:"2",
      "name": "Camera 2",
      "privilege": 5,
      "settings": {
        "PRESET.name.count": "0",
        "PRESET.preset.count": "0",
        "TOUR.name.count": "0",
        "TOUR.tour.count": "0",
        "advanced": "{\"_stream2\":0,\"_stream3\":0,\"digest_auth_only\":0,\"edge
\":1,\"edge_skip_last_minutes\":10,\"fisheye_lens\":null,\"fisheye_position
\":0,\"over_rtsp1\":0,\"over_rtsp2\":0,\"over_rtsp3\":0,\"profile1\":null,\"profile2
\":null,\"profile3\":null,\"rtp_network_latency1\":500,\"rtp_network_latency2
\":500,\"rtp_network_latency3\":500,\"speaker\":null,\"speaker_ip
\":\"\",\"speaker_model\":\"Axis Network Speaker\",\"speaker_password
\":\"EJABDJPACKPDAFNH\",\"speaker_type\":\"Axis\",\"speaker_user\":\"\",\"strm_arch
\":0,\"strm_high\":0,\"strm_low\":0,\"strm_md\":0,\"strm_norm\":0,\"tour_type\":\"0
\",\"use_default_video1\":1,\"use_default_video2\":1,\"use_default_video3\":1,\"washing
\":null}",
        "alarm_rec": "1",

       ... // Some parameters are excluded

        "short_exposures": "",
        "telemetry_id": "",
        "wiper": ""
      },
      "status": {
```

```
        "enabled": true,
        "focused": "true",
        "hold_ptz_SLAVE_ID": "",
        "hold_ptz_module_id": "",
        "hold_ptz_priority": "",
        "hold_ptz_user_id": "",
        "priority": "-1",
        "recorded": "0",
        "state": "DISARMED",
        "unblinded": "true",
        "valid": true
      },
      "type": "CAM"
    },
    {
      “id”:"1",
      "name": "Camera 1",
      "privilege": 5,
      "settings": {
        "PRESET.name.count": "0",
        "PRESET.preset.count": "0",

       ... // Some parameters are excluded

        "short_exposures": "",
        "telemetry_id": "",
        "wiper": ""
      },
      "status": {
        "enabled": true,
        "focused": "true",
        "hold_ptz_SLAVE_ID": "",
        "hold_ptz_module_id": "",
        "hold_ptz_priority": "",
        "hold_ptz_user_id": "",
        "priority": "",
        "recorded": "0",
        "state": "DISARMED",
        "unblinded": "true",
        "valid": true
      },
      "type": "CAM"
    }
  ],
  "status": "success"
}
```


General response parameters

Parameter	Description
Root parameters	

Parameter	Description
`privilege`	The user right for *Camera* objects listed in the response granted to the user who sent the request. May have one of the following values (see **SecurOS Administration Guide** for more information): • `0` — **no access** (). • `1` — **view** (). • `2` — **control** (). • `4` — **configure** (). • `5` — **full access** ().
"settings" section parameters	
`audio_id`	ID of the *Microphone* linked to this *Camera*. May have one of the following values: • `<empty string>` — there is no *Microphone* linked to the *Camera*. • `<other value>` — identifier of the *Microphone* linked to the *Camera*.
`telemetry_id`	*Camera* PTZ device attribute. Corresponds with **Pan/Tilt/Zoom** parameter of *Camera* settings (**General** tab). May have one of the following values: • `<empty strings>` — no PTZ device. • `native` — built-in PTZ device is used. • `<other value>` — identifier of the device selected as *PTZ controller* and *Pan/Tilt/Zoom* (obsolete feature).
`wiper`	The attribute of *Camera Wiper*. Corresponds with **Wiper** parameter of *Camera* settings (**General** tab). May have one of the following values: • `<empty strings>` — no *Wiper*. • `0` — built-in *Wiper* is used. • `<other value>` — identifier of the *Relay* object selected as *Wiper*.
"status" section parameters	
`enabled`	State of the *Camera* in the SecurOS *Object tree*. May have one of the following values: • `true` — *Camera* is enabled. • `false` — *Camera* is disabled.

Parameter	Description
`focused`	Focus of the *Camera* (defined by state of the *Defocus detector*). May have one of the following values: • `true` — *Camera* is focused. • `false` — *Camera* is unfocused.
`recorded`	*Camera* recording attribute. May have one of the following values: • `0` — recording is disabled. • `1` — recording is enabled. • `<any negative number>` — recording is enabled but there is an error occurred.
`state`	*Camera* current state. May have one of the following values: • `ALARMED` — alarm (Motion detector is triggered). • `ARMED` — Camera is armed. • `DETACHED` — no signal from the Camera. • `DISARMED` — Camera is disarmed. • `DISARM_DETACHED` — no signal from the Camera.
`unblinded`	Blinded state of the *Camera* (defined by state of *Blinding detector*). May have one of the following values: • `true` — *Camera* is not blinded. • `false` — *Camera* is blinded.

6.2 Get State of Specified Camera

Returns the state of specific *Camera*.

Resource ID

```
/api/v1/cameras/<cam_id>/status
```

Objects Access Rights

Camera — `View` (👁) and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/cameras/1/status
```

Response Example


```
{
  "data": {
    "id": "1",
    "name": "Camera 1",
    "privilege": 5,
    "status": {
      "enabled": true,
      "focused": "1",
      "hold_ptz_SLAVE_ID": "",
      "hold_ptz_module_id": "",
      "hold_ptz_priority": "",
      "hold_ptz_user_id": "",
      "lock": "",
      "priority": "",
      "recorded": "0",
      "state": "DISARM_DETACHED",
      "unblinded": "1",
      "valid": false
    },
    "type": "CAM"
  },
  "status": "success"
}
```


General response parameters

Parameter	Description
Root parameters	
`privilege`	The user right for the specified *Camera* object granted to the user who sent the request. May have one of the following values (see **SecurOS Administration Guide** for more information): • `0` — **no access** (✕). • `1` — **view** (👁). • `2` — **control** (✋). • `4` — **configure** (⚙). • `5` — **full access** (🎬).
"status" section parameters	

Parameter	Description
`enabled`	State of the *Camera* in the SecurOS *Object tree*. May have one of the following values: • `true` — *Camera* is enabled. • `false` — *Camera* is disabled.
`focused`	Focus of the *Camera* (defined by state of the *Defocus detector*). May have one of the following values: • `true` — *Camera* is focused. • `false` — *Camera* is unfocused.
`recorded`	*Camera* recording attribute. May have one of the following values: • `0` — recording is disabled. • `1` — recording is enabled. • `<any negative number>` — recording is enabled but there is an error occurred.
`state`	*Camera* current state. May have one of the following values: • `ALARMED` — alarm (Motion detector is triggered). • `ARMED` — Camera is armed. • `DETACHED` — no signal from the Camera. • `DISARMED` — Camera is disarmed. • `DISARM_DETACHED` — no signal from the camera.
`unblinded`	Blinded state of the *Camera* (defined by state of *Blinding detector*). May have one of the following values: • `true` — *Camera* is not blinded. • `false` — *Camera* is blinded.

6.3 Get Camera Connection Settings

Returns the *Camera* connection settings and state.

Resources IDs

- `/api/v2/cameras` — to get connection settings and state of all *Cameras* of configuration as array.
- `/api/v2/cameras/<cam_id>` — to get connection settings and state of specific *Camera*.

Objects Access Rights

Camera — `View` (👁) and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

`HTTP/GET`

Request Example

`http://172.16.1.170:8888/api/v2/cameras`

Response Example


```
{
  "data": [
    {
      “id”:"2",
      "integrator": "Axis",
      "login": "root",
      "model": "P3215-V",
      "name": "Camera 2",
      "privilege": 5,
      "server": "VIDEO_SERVER",
      "settings": {
        "PRESET.name.count": "0",
        "PRESET.preset.count": "0",
        "TOUR.name.count": "0",
        "TOUR.tour.count": "0",
        "telemetry_id": ""
      },
      "status": {
        "enabled": true,
        "focused": "1",
        "hold_ptz_SLAVE_ID": "",
        "hold_ptz_module_id": "",
        "hold_ptz_priority": "",
        "hold_ptz_user_id": "",
        "lock": "",
        "priority": "",
        "recorded": "0",
        "state": "DISARMED",
        "unblinded": "1",
        "valid": true
      "url": "172.16.20.54"
    },
    {
      “id”:"1",
      "integrator": "Virtual",
      "login": "",
      "model": "default",
      "name": "Camera 1",
      "privilege": 5,
      "server": "VIDEO_SERVER",
```

```
        "settings": {
          "PRESET.name.count": "0",
          "PRESET.preset.count": "0",
          "TOUR.name.count": "0",
          "TOUR.tour.count": "0",
          "telemetry_id": ""
        },
        "status": {
          "enabled": true,
          "focused": "1",
          "hold_ptz_SLAVE_ID": "",
          "hold_ptz_module_id": "",
          "hold_ptz_priority": "",
          "hold_ptz_user_id": "",
          "lock": "",
          "priority": "",
          "recorded": "0",
          "state": "DISARMED",
          "unblinded": "1",
          "valid": true
        },
        "url": ""
      }
    ],
    "status": "success"
  }
```


General response parameters

Parameter	Description
Root parameters	
`privilege`	The user right for *Camera* objects listed in the response granted to the user who sent the request. May have one of the following values (see **SecurOS Administration Guide** for more information): • `0` — **no access** (✕). • `1` — **view** (👁). • `2` — **control** (✋). • `4` — **configure** (⚙). • `5` — **full access** (🎬).
"settings" section parameters	
`telemetry_id`	*Camera* PTZ device attribute. Corresponds with **Pan/Tilt/Zoom** parameter of *Camera* settings (**General** tab). May have one of the following values: • `<empty strings>` — no PTZ device. • `native` — built-in PTZ device is used. • `<other value>` — identifier of the device selected as *PTZ controller* and *Pan/Tilt/Zoom* (obsolete feature).
"status" section parameters	

Parameter	Description
`enabled`	State of the *Camera* in the SecurOS *Object tree*. May have one of the following values: • `true` — *Camera* is enabled. • `false` — *Camera* is disabled.
`focused`	Focus of the *Camera* (defined by state of the *Defocus detector*). May have one of the following values: • `true` — *Camera* is focused. • `false` — *Camera* is unfocused.
`recorded`	*Camera* recording attribute. May have one of the following values: • `0` — recording is disabled. • `1` — recording is enabled. • `<any negative number>` — recording is enabled but there is an error occurred.
`state`	*Camera* current state. May have one of the following values: • `ALARMED` — alarm (Motion detector is triggered). • `ARMED` — Camera is armed. • `DETACHED` — no signal from the Camera. • `DISARMED` — Camera is disarmed. • `DISARM_DETACHED` — no signal from the Camera.
`unblinded`	Blinded state of the *Camera* (defined by state of *Blinding detector*). May have one of the following values: • `true` — *Camera* is not blinded. • `false` — *Camera* is blinded.

6.4 Change Connection Settings of Specified Camera/Rename Camera

Changes connection settings and name of the *Camera* and shows the new values.

Resource ID

```
/api/v2/cameras/<cam_id>
```

Objects Access Rights

Camera — `Configure` (⚙) and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

`HTTP/PUT`

Required Header

```
Content-Type: application/json
```

Request Body

The new *Camera* connection setting or the new *Camera* name must be passed in the request. If parameter is not specified in the request it will keep its current value.

Request Body Example

```
{
    "integrator" : "Panasonic",
    "model" : "WV-SF342",
    "login" : "admin",
    "password": "admin",
    "name" : "New_camera_name",
    "url" : "172.16.1.170"
}
```

Request Example

```
http://172.16.1.170:8888/api/v2/cameras/7
```

Response Example


```
{
  "data": {
    "id": "7",
    "integrator": "Panasonic",
    "login": "admin",
    "model": "WV-SF342",
    "name": "New_camera_name",
    "server": "VIDEO_SERVER_II",
    "settings": {
      "PRESET.name.count": "0",
      "PRESET.preset.count": "0",
      "TOUR.name.count": "0",
      "TOUR.tour.count": "0",
      "telemetry_id": ""
    },
    "status": {
      "enabled": true,
      "focused": "1",
      "hold_ptz_SLAVE_ID": "",
      "hold_ptz_module_id": "",
      "hold_ptz_priority": "",
      "hold_ptz_user_id": "",
      "lock": "",
```

```
        "priority": "",
        "recorded": "0",
        "state": "DISARM_DETACHED",
        "unblinded": "1",
        "valid": false
      },
      "url": "172.16.1.170"
    },
    "status": "success"
  }
```


General response parameters

Parameter	Description
Root parameters	
`privilege`	The user right for the specified *Camera* object granted to the user who sent the request. May have one of the following values (see **SecurOS Administration Guide** for more information): • `0` — **no access** (✕). • `1` — **view** (👁). • `2` — **control** (✋). • `4` — **configure** (⚙). • `5` — **full access** (🎬).
"settings" section parameters	
`telemetry_id`	*Camera* PTZ device attribute. Corresponds with **Pan/Tilt/Zoom** parameter of *Camera* settings (**General** tab). May have one of the following values: • `<empty strings>` — no PTZ device. • `native` — built-in PTZ device is used. • `<other value>` — identifier of the device selected as *PTZ controller* and *Pan/Tilt/Zoom* (obsolete feature).
"status" section parameters	
`enabled`	State of the *Camera* in the SecurOS *Object tree*. May have one of the following values: • `true` — *Camera* is enabled. • `false` — *Camera* is disabled.
`focused`	Focus of the *Camera* (defined by state of the *Defocus detector*). May have one of the following values: • `true` — *Camera* is focused. • `false` — *Camera* is unfocused.

Parameter	Description
`recorded`	*Camera* recording attribute. May have one of the following values: • `0` — recording is disabled. • `1` — recording is enabled. • `<any negative number>` — recording is enabled but there is an error occurred.
`state`	*Camera* current state. May have one of the following values: • `ALARMED` — alarm (Motion detector is triggered). • `ARMED` — Camera is armed. • `DETACHED` — no signal from the Camera. • `DISARMED` — Camera is disarmed. • `DISARM_DETACHED` — no signal from the Camera.
`unblinded`	Blinded state of the *Camera* (defined by state of *Blinding detector*). May have one of the following values: • `true` — *Camera* is not blinded. • `false` — *Camera* is blinded.

6.5 Delete Camera

Resource ID

`/api/v2/cameras/<cam_id>`

Objects Access Rights

Camera — `Full access` ().

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

`HTTP/DELETE`

Request Example

```
http://172.16.1.170:8888/api/v2/cameras/5
```

Response Example

```
{
    "status": "success"
}
```

Warning! If the *Camera* that is being deleted is the only one child of the *Video Capture Device*, then this *Video Capture Device* will also be deleted along with the *Camera*.

6.6 Working With PTZ Camera

The following requests are provided for working with the *Camera*'s PTZ device:

- **Rotate/Scale Camera**.
- **Rotate/Scale Camera Using Absolute Coordinates**.
- **Get PTZ Camera State**.
- **Get Tour and Preset List**.
- **Go To Preset**.
- **Start Patrol**.
- **Stop Patrol**.

6.6.1 Rotate/Scale Camera

Resource ID

```
/api/v1/cameras/<cam_id>/ptz/move?pan_speed=<value1>&tilt_speed=<value2>
&zoom_speed=<value3>&duration=<value4>
```

or

```
/api/v2/cameras/<cam_id>/ptz/move?pan_speed=<value1>&tilt_speed=<value2>
&zoom_speed=<value3>&duration=<value4>
```

Objects Access Rights

Camera — `Control` () and higher.

Request Parameters

- **cam_id** — identifier of the controlled *Camera*;
- **pan_speed** — horizontal rotation speed. Possible values: [`-10; 10`] (on the left; on the right);
- **tilt_speed** — vertical rotation speed. Possible values: [`-10; 10`] (up; down);
- **zoom_speed** — scaling speed. Possible values: [`-10; 10`] (zoom out; zoom in);
- **duration** — time after which the movement in the specified directions and scaling will be stopped (in milliseconds).

Protocol/Method

```
HTTP/POST
```

Request Example

```
http://172.16.1.170:8888/api/v1/cameras/8/ptz/move?pan_speed=4&tilt_speed=0
&zoom_speed=0
```

Warning! If the **duration** is not set then rotation in specified direction and scaling is carried out until:

1. A command with zero parameter values is received (`pan_speed=0`, `tilt_speed=0` and `zoom_speed=0`).
2. Thresholds of the camera tilt, pan or zoom are reached. In this case, the camera will be stopped automatically.

Response Example

```
{
    "status": "success"
}
```

6.6.2 Rotate/Scale Camera Using Absolute Coordinates

Note. The resource is only available for some camera models.

Resource ID

```
/api/v1/cameras/<cam_id>/ptz/move_abs?pan_position=<value1>
&tilt_position=<value2>&zoom_position=<value3>
```

or

```
/api/v2/cameras/<cam_id>/ptz/move_abs?pan_position=<value1>
&tilt_position=<value2>&zoom_position=<value3>
```

Objects Access Rights

Camera — `Control` () and higher.

Request Parameters

- **cam_id** — identifier of the controlled *Camera*;
- **pan_position** — horizontal coordinates. Possible values: [`-10800; 10800`] (on the left; on the right);
- **pan_position** — vertical coordinates. Possible values: [`-5400; 5400`] (up; down);
- **zoom_position** — scaling. Possible values: [`0; 1000`] (zoom out; zoom in).

Protocol/Method

```
HTTP/POST
```

Request Example

```
http://172.16.1.170:8888/api/v1/cameras/8/ptz/move_abs?pan_position=7250
&tilt_position=5000&zoom_position=240
```

Response Example

```
{
    "status": "success"
}
```

6.6.3 Get PTZ Camera State

Note. The resource is only available for some camera models.

Resource ID

```
/api/v1/cameras/<cam_id>/ptz/status?timeoutMs=<value>
```

or

```
/api/v2/cameras/<cam_id>/ptz/status?timeoutMs=<value>
```

Objects Access Rights

Camera — `View` (👁) and higher.

Request Parameters

- **cam_id** — identifier of the controlled *Camera*;
- **timeoutMs** — server response timeout, in milliseconds. Optional parameter. If not specified, will be set to `1000`. If specified value is less than `20` will be set to `20`.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/cameras/8/ptz/status?timeoutMs=500
```

Response Example

```
{
  "data": {
    "pan_position": "-3242",
    "pantilt_status": "IDLE",
    "tilt_position": "-2272",
    "zoom_position": "1000",
    "zoom_status": "IDLE"
  },
  "status": "success"
}
```

6.6.4 Get Tour and Preset List

Resource ID

```
api/v1/cameras/<cam_id>/ptz?timeoutMs=<value>
```

or

```
api/v2/cameras/<cam_id>/ptz?timeoutMs=<value>
```

Objects Access Rights

Not required.

Request Parameters

- **cam_id** — identifier of the requested *Camera*;
- **timeoutMs** — server response timeout, in milliseconds. Optional parameter. If not specified, will be set to `1000`. If specified value is less than `20` will be set to `20`.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/cameras/8/ptz?timeoutMs=500
```

Response Example

```
{
    "data":[
       “presets":[
          {
             “name":"Main entrance",
             “id":"1",
          },
          {
             “name":"Stairs",
             “id":"2",
          }
       ],
        “tours":[
          {
             “name":"tour1",
             “id":"1",
          },
          {
             “name":"Long patrol",
             “id":"rpt_1",
          }
       ],
   ],
    "status": "success"
}
```

6.6.5 Go To Preset

Resource ID

```
/api/v1/cameras/<cam_id>/ptz/presets/recall?preset=<preset_id>
&pt_speed=<value>
```

or

```
/api/v2/cameras/<cam_id>/ptz/presets/recall?preset=<preset_id>
&pt_speed=<value>
```

Objects Access Rights

Camera — `Control` () and higher.

Request Parameters

- **cam_id** — identifier of the controlled *Camera*;
- **preset** — identifier of the *Preset* on the *Camera*;
- **pt_speed** — speed of the *Camera* move. Possible values: [`0; 10`].

Note. The **pt_speed** parameter is supported only by some camera models.

Protocol/Method

```
HTTP/POST
```

Request Example

```
http://172.16.1.170:8888/api/v2/cameras/8/ptz/presets/recall?preset=3
&pt_speed=7
```

Response Example

```
{
    "status": "success"
}
```

6.6.6 Start Patrol

Resource ID

```
/api/v1/cameras/<cam_id>/ptz/patrols/play?patrol=<patrol_id>
```

or

```
/api/v2/cameras/<cam_id>/ptz/patrols/play?patrol=<patrol_id>
```

Objects Access Rights

Camera — `Control` () and higher.

Request Parameters

- **cam_id** — identifier of the controlled *Camera*;
- **patrol** — identifier of the *Tour* on the *Camera*.

Protocol/Method

`HTTP/POST`

Request Example

```
http://172.16.1.170:8888/api/v2/cameras/8/ptz/patrols/play?patrol=1
```

Response Example

```
{
    "status": "success"
}
```

6.6.7 Stop Patrol

Resource ID

`/api/v1/cameras/<cam_id>/ptz/patrols/stop`

or

`/api/v2/cameras/<cam_id>/ptz/patrols/stop`

Objects Access Rights

Camera — `Control` () and higher.

Request Parameters

cam_id — identifier of the controlled *Camera*.

Protocol/Method

`HTTP/POST`

Request Example

```
http://172.16.1.170:8888/api/v2/cameras/8/ptz/patrols/stop
```

Response Example

```
{
    "status": "success"
```

```
}
```

6.7 Camera Recording Control

The following requests are provided for *Camera* recording control:

- **Start Record**.
- **Start Record and Add Pre-recorded Video from Pre-recording Buffer**.
- **Stop Record**.

6.7.1 Start Record

Resource ID

```
/api/v1/cameras/<cam_id>/recording/rec
```

or

```
/api/v2/cameras/<cam_id>/recording/rec
```

Objects Access Rights

Camera — `Control` () and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

```
HTTP/POST
```

Request Example

```
http://172.16.1.170:8888/api/v2/cameras/1/recording/rec
```

Response Example

```
{
    "status": "success"
}
```

6.7.2 Start Record and Add Pre-recorded Video from Pre-recording Buffer

Resource ID

```
/api/v1/cameras/<cam_id>/recording/rec_rollback?rollback_time_abs=<time>
```

or

```
/api/v2/cameras/<cam_id>/recording/rec_rollback?rollback_time_abs=<time>
```

Objects Access Rights

Camera — `Control` () and higher.

Request Parameters

- **cam_id** — identifier of the *Camera*.
- **rollback_time_abs** — absolute time on the time line for the current date in the `HH:MM:SS.FFF` (milliseconds are optional). For more information about the parameter, see the **SecurOS Programming Guide**.

Protocol/Method

`HTTP/POST`

Request Example

```
http://172.16.1.170:8888/api/v2/cameras/1/recording/rec_rollback
?rollback_time_abs=10:08:14
```

Response Example

```
{
    "status": "success"
}
```

6.7.3 Stop Record

Resource ID

`/api/v1/cameras/<cam_id>/recording/rec_stop`

or

`/api/v2/cameras/<cam_id>/recording/rec_stop`

Objects Access Rights

Camera — `Control` () and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

`HTTP/POST`

Request Example

`http://172.16.1.170:8888/api/v2/cameras/1/recording/rec_stop`

Response Example


```
{
    "status": "success"
}
```


6.8 Working with Cameras of RTSP Server

The following requests are provided for working with the *Cameras* of the RTSP Server:

- **Get Information about RTSP Streams of Camera**.
- **Get Information about Live Video RTSP Stream of Camera**.
- **Get Information about Archive Video RTSP Stream of Camera**.

Note. For the details on the RTSP server, RTSP camera streams, and RTSP requests, see the **SecurOS Administration Guide**.

6.8.1 Get Information about RTSP Streams of Camera

Resource ID

`/api/v1/cameras/<cam_id>/rtsp`

or

`/api/v2/cameras/<cam_id>/rtsp`

Objects Access Rights

Camera — `View` (👁) and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

`HTTP/GET`

Request Example

`http://172.16.1.170:8888/api/v2/cameras/4/rtsp`

Response Example


```
{
  "data": {
    "id": "4",
    "rtsp": [
      {
        "type": "live",
        "url": "rtsp://172.16.1.170:554/live?id=4"
```

```
      },
      {
        "type": "archive",
        "url": "rtsp://172.16.1.170:554/archive?id=4"
      }
    ],
    "type": "CAM"
  },
  "status": "success"
}
```


The following parameters are returned in the response:

- **rtsp.type** — type of the *Camera*'s RTSP stream;
- **rtsp.url** — URL to receive the *Camera*'s RTSP stream.

6.8.2 Get Information about Live Video RTSP Stream of Camera

Resource ID

```
/api/v1/cameras/<cam_id>/rtsp/live
```

or

```
/api/v2/cameras/<cam_id>/rtsp/live
```

Objects Access Rights

Camera — `View` (👁) and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v2/cameras/4/rtsp/live
```

Response Example


```
{
  "data": {
    "id": "4",
    "rtsp": [
      {
        "type": "live",
        "url": "rtsp://172.16.1.170:554/live?id=4"
      }
    ],
    "type": "CAM"
  },
  "status": "success"
```

```
}
```

The following parameters are returned in the response:

- **rtsp.type** — type of the *Camera*'s RTSP stream;
- **rtsp.url** — URL to receive the live *Camera*'s RTSP stream.

6.8.3 Get Information about Archive Video RTSP Stream of Camera

Resource ID

```
/api/v1/cameras/<cam_id>/rtsp/archive
```

or

```
/api/v2/cameras/<cam_id>/rtsp/archive
```

Objects Access Rights

Camera — `View` (👁) and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v2/cameras/4/rtsp/archive
```

Response Example


```
{
  "data": {
    "id": "4",
    "rtsp": [
      {
        "type": "archive",
        "url": "rtsp://172.16.1.170:554/archive?id=4"
      }
    ],
    "type": "CAM"
  },
  "status": "success"
}
```


The following parameters are returned in the response:

- **rtsp.type** — type of the *Camera*'s RTSP stream;
- **rtsp.url** — URL to receive the archive *Camera*'s RTSP stream.

6.9 Working with Washer and Wiper

The following requests are provided for working with the *Camera*'s Washer and Wiper:

- **Start Wiper**.
- **Start Washing**.

6.9.1 Start Wiper

Resource ID

`/api/v1/cameras/<cam_id>/ptz/wiper`

or

`/api/v2/cameras/<cam_id>/ptz/wiper`

Objects Access Rights

Camera — `Control` () and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

`HTTP/POST`

Request Example

`http://172.16.1.170:8888/api/v1/cameras/12/ptz/wiper`

Response Example

```
{
    "status": "success"
}
```

6.9.2 Start Washing

Resource ID

`/api/v1/cameras/<cam_id>/ptz/washing`

or

`/api/v2/cameras/<cam_id>/ptz/washing`

Objects Access Rights

Camera — `Control` () and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

`HTTP/POST`

Request Example

```
http://172.16.1.170:8888/api/v1/cameras/12/ptz/washing
```

Response Example

```
{
    "status": "success"
}
```

7 Video Archive Export

The following requests are provided for working with the video archive export tasks:

- **Create Archive Export Task**.
- **Get Information about All Current Archive Export Tasks**.
- **Get Information about Specified Archive Export Task**.

7.1 Create Archive Export Task

Note. The task will be performed by that *Archive Converter* of this *Video Server*, which has the lowest identifier.

Resource ID

```
/api/v1/export/tasks
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

```
HTTP/POST
```

Required Header

```
Content-Type: application/json
```

Request Body

The following parameters are passed in the request body:

- **cam_id** — identifier of the *Camera* whose archive fragment needs to be exported;
- **from**, **to** — left and right border of the requested time interval, in the `YYYYMMDD`**`T`**`HHMMSS[.SSS]` format (milliseconds are optional).

Request Body Example

```
{
    "camera":"2",
    "from":"20200219T090440.000",
    "to":"20200219T092040.000"
```

```
}
```

Request Example

```
http://172.16.1.170:8888/api/v1/export/tasks
```

Response Example


```
{
  "data": {
    "camera": "2",
    "create_time": "2020-02-20T11:59:48.887",
    "from": "2020-02-19T09:04:40.000",
    "id": "412dc56f-73f7-4512-9a5a-c9d72a126a72",
    "state": "Registered",
    "to": "2020-02-19T09:20:40.000"
  },
  "status": "success"
}
```


7.2 Get Information about All Current Archive Export Tasks

Resource ID

```
/api/v1/export/tasks
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/export/tasks
```

Response Example


```
{
  "data": [
    {
      "camera": "3",
      "create_time": "2020-02-20T12:01:04.110",
      "from": "2020-02-19T09:04:40.000",
      "id": "2f5a37a9-684f-4377-b43e-81ba13be8e25",
      "state": "Registered",
      "to": "2020-02-19T10:20:40.000"
    },
    {
```

```
      "actual_archive_start_time": "2020-02-19T09:04:40.009",
      "actual_archive_stop_time": "2020-02-19T09:20:39.971",
      "camera": "1",
      "create_time": "2020-02-20T11:59:48.887",
      "from": "2020-02-19T09:04:40.000",
      "id": "412dc56f-73f7-4512-9a5a-c9d72a126a72",
      "operation": "Converting archive",
      "percents": 100,
      "state": "Finished",
      "to": "2020-02-19T09:20:40.000",
      "total_progress": 100,
      "url": "\\412dc56f-73f7-4512-9a5a-c9d72a126a72.ASF"
    },
    {
      "camera": "2",
      "create_time": "2020-02-20T12:12:47 AM.581",
      "from": "2020-02-19T09:04:40.000",
      "id": "4db57466-f80a-4632-b693-3117542cb54f",
      "operation": "Converting archive",
      "percents": 39,
      "state": "Started",
      "to": "2020-02-19T10:00:40.000",
      "total_progress": 51
    },
    {
      "camera": "5",
      "create_time": "2020-02-20T12:12:22 PM.274",
      "from": "2020-02-19T09:04:40.000",
      "id": "ba26860f-03e4-4af6-8e09-99d5574dd372",
      "state": "Registered",
      "to": "2020-02-19T10:20:40.000"
    }
  ],
  "status": "success"
}
```


Possible Export Tasks States

Code	Description
Registered	Task created
Enqueued	Task added to the *Archive Converter* task queue
Started	*Archive Converter* has started the task
Finished	Task finished successfully
Failed	Task failed
Cancelled	Task canceled by user

7.3 Get Information about Specified Archive Export Task

Resource ID

```
/api/v1/export/tasks/<export_task_id>
```

Objects Access Rights

Not required.

Request Parameters

export_task_id — identifier of the export task assigned when it has been created.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/export/tasks/
ba26860f-03e4-4af6-8e09-99d5574dd372
```

Response Example


```
{
  “data": {
    "camera": "5",
    "create_time": "2020-02-20T12:12:22 PM.274",
    "error": "LoadingError",
    "from": "2020-02-19T09:04:40.000",
    "id": "ba26860f-03e4-4af6-8e09-99d5574dd372",
    "problem": "It's impossible to receive archive (no fragments on servers), hosts:
'VIDEO_SERVER_II, '",
    "state": "Failed",
    "to": "2020-02-19T10:20:40.000"
  },
  "status": "success"
}
```

8 Frame Export

The following requests allow to export frames:

- **Get Frame of Live Video from Specified Camera**.
- **Get Frame of Archive Video from Specified Camera**.
- **Get Frame of Specified Resolution from Specified Camera**.

Warning! To export frames the *Image Processor* object must be created in SecurOS (see **SecurOS Administration Guide**). Otherwise, request will fail.

8.1 Get Frame of Live Video from Specified Camera

Resource ID

```
/api/v2/cameras/<cam_id>/image
```

Objects Access Rights

Camera — `View` (👁) and higher.

Request Parameters

cam_id — identifier of the *Camera*.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v2/cameras/1/image
```

Image is sent to the REST API client in the response body and further can be saved to a file. The examples of the request headers and response body are presented below. Received image is conventionally designated as `[DATA]`.

Response Example

```
Server: Microsoft-HTTPAPI/2.0
Cache-Control: no-store, must-revalidate
Content-Type: image/jpeg
Transfer-Encoding: chunked
Date: Fri, 21 Feb 2020 07:02:07 GMT

[DATA]
```

8.2 Get Frame of Archive Video from Specified Camera

Resource ID

`/api/v2/cameras/<cam_id>/image/<frame_time>`

Objects Access Rights

Camera — `View` (👁) and higher.

Request Parameters

- **cam_id** — identifier of the *Camera* whose archive frame needs to be exported;
- **frame_time** — frame timestamp in one of the following formats (milliseconds are optional):
  - `YYYYMMDD`**`T`**`HHMMSS[.SSS];`
  - `YYYY-MM-DD`**`T`**`HH:MM:SS[.SSS];`
  - `YYYY-MM-DD%20HH:MM:SS[.SSS].`

Protocol/Method

`HTTP/GET`

Request Example

`http://172.16.1.170:8888/api/v2/cameras/2/image/20200219T091259`

Image is sent to the REST API client in the response body and further can be saved to a file. The examples of the request headers and response body are presented below. Received image is conventionally designated as `[DATA]`.

Response Example

```
Server: Microsoft-HTTPAPI/2.0
Cache-Control: no-store, must-revalidate
Content-Type: image/jpeg
Transfer-Encoding: chunked
Date: Fri, 21 Feb 2020 07:02:07 GMT

[DATA]
```

8.3 Get Frame of Specified Resolution from Specified Camera

Resource ID

`/api/v2/cameras/<cam_id>/image?scale_y=<pixels>&scale_x=<pixels>` - for the live video

or

`/api/v2/cameras/<cam_id>/image/<frame_time)>?scale_y=<pixels>&scale_x=<pixels>` - for the archive video

Objects Access Rights

Camera — `View` (👁) and higher.

Request Parameters

- **cam_id** — identifier of the *Camera* whose frame needs to be exported;
- **scale_y** — vertical image size (in pixels);
- **scale_x** — horizontal image size (in pixels);
- **frame_time** — frame timestamp in one of the following formats (milliseconds are optional):
  - `YYYYMMDD`**`T`**`HHMMSS[.SSS];`
  - `YYYY-MM-DD`**`T`**`HH:MM:SS[.SSS];`
  - `YYYY-MM-DD%20HH:MM:SS[.SSS].`

Protocol/Method

`HTTP/GET`

Request Example

```
http://172.16.1.170:8888/api/v2/cameras/2/image/20200219T091259?scale_y=400&scale_x=300
```

Image is sent to the REST API client in the response body and further can be saved to a file. The examples of the request headers and response body are presented below. Received image is conventionally designated as `[DATA]`.

Response Example

```
Server: Microsoft-HTTPAPI/2.0
Cache-Control: no-store, must-revalidate
Content-Type: image/jpeg
Transfer-Encoding: chunked
Date: Fri, 21 Feb 2020 07:02:07 GMT

[DATA]
```

9 Working with User Accounts

The following requests are provided for working with SecurOS *User Accounts*:

- **Get Information about All Departments**.
- **Get Information About Specified Department**.
- **Get Data of All User Accounts**.
- **Get Data of Specified User Account**.
- **Create User Account**.
- **Change User Account Settings**.
- **Delete User Account**.

9.1 Get Information about All Departments

Resource ID

```
/api/v1/departments
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/departments
```

Response Example


```
{
  "data": [
    {
      "id": "1.1",
      "name": "Department 1"
    },
    {
      "id": "1.2",
      "name": "Department 2"
    }
```

```
  ],
  "status": "success"
}
```

9.2 Get Information about Specified Department

Resource ID

```
/api/v1/departments/<department_id>
```

Objects Access Rights

Not required.

Request Parameters

department_id — identifier of the *Department*.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/departments/1.2
```

Response Example

```
{
  "data": {
    "id": "1.2",
    "name": "Department 2"
  },
  "status": "success"
}
```

9.3 Get Data of All User Accounts

Resource ID

```
/api/v1/persons
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

HTTP/GET

Request Example

http://172.16.1.170:8888/api/v1/persons

Response Example


```
{
  "data": [
    {
      "comment": "",
      "department_id": "1.1",
      "email": "a@b.c,
      "id": "1.1",
      "name": "Operator_1",
      "phone": "55555555",
      "user_rights_id": "1.3"
    },
    {
      "comment": "Timetable: Wed-Fri from 9-00 AM till 4-00 PM",
      "department_id": "1.1",
      "email": "a@b.c",
      "id": "1.2",
      "name": "Operator_2",
      "phone": "5555555",
      "user_rights_id": "1.3"
    },
  ],
  "status": "success"
}
```


9.4 Get Data of Specified User Account

Resource ID

/api/v1/persons/<person_id>

Objects Access Rights

Not required.

Request Parameters

person_id — identifier of the *User Account*.

Protocol/Method

HTTP/GET

Request Example

```
http://172.16.1.170:8888/api/v1/persons/1.1
```

Response Example


```
{
  "data": {
    "comment": "",
    "department_id": "1.1",
    "email": "a@b.c",
    "id": "1.1",
    "name": "Operator_1",
    "phone": "5555555",
    "user_rights_id": "1.3"
  },
  "status": "success"
}
```


9.5 Create User Account

Resource ID

```
/api/v1/persons
```

Objects Access Rights

- **Department** — `Full access` ();
- **User Rights** — `Configure` () and higher.

Protocol/Method

```
HTTP/POST
```

Request Parameters

None.

Required Header

```
Content-Type: application/json
```

Request Body

In the request body one must pass the credentials of a new SecurOS *User Account*, identifier of the parent *Department* and identifier of the assigned *User Rights*. If parameter is not specified in the request it will be empty.

Request Body Example


```
{
  "department_id": "1.2",
  "name": "Operator 10",
  "password": "pwd",
```

```
  "phone": "88003222334",
  "email": "email@mail",
  "comment": "custom_comment",
  "user_rights_id": "1.3"
}
```

Warning! `department_id`, `name` and `user_rights_id` parameters are mandatory!

Request Example

```
http://172.16.1.170:8888/api/v1/persons
```

Response Example

```
{
  “data": {
    "comment": "custom_comment",
    "department_id": "1.2",
    "email": "email@mail",
    “id":"8",
    "name": "Operator 10",
    "phone": "88003222334",
    "user_rights_id": "1.3"
  },
  "status": "success"
}
```

9.6 Change User Account Settings

Resource ID

```
/api/v1/persons/<person_id>
```

Objects Access Rights

Department — `Configure` (⚙) and higher.

Request Parameters

person_id — identifier of the *User Account*.

Protocol/Method

```
HTTP/PUT
```

Required Header

```
Content-Type: application/json
```

Request Body

In the request body one must pass new credentials of the SecurOS *User Account* and identifier of the new *User Rights*. If parameter is not specified in the request it will keep its current value.

Request Body Example

```
Content-Type: application/json

{
  "comment": "new_custom_comment",
  "email": "e@mail",
  "name": "Operator 5",
  "phone": "88003222334",
  "password": "15",
  "user_rights_id": "1.2"
}
```

Warning! `department_id` and `user_rights_id` parameters are mandatory!

Request Example

```
http://172.16.1.170:8888/api/v1/persons/1.5
```

Response Example

```
{
  "data": {
    "comment": "new_custom_comment",
    "department_id": "1.2",
    "email": "e@mail",
    "id": "1.5",
    "name": "Operator 5",
    "phone": "88003222334",
    "user_rights_id": "1.2"
  },
  "status": "success"
}
```

9.7 Delete User Account

Resource ID

```
/api/v1/persons/<person_id>
```

Objects Access Rights

Department — `Full control` (🎬).

Request Parameters

person_id — identifier of the *User Account*.

Protocol/Method

```
HTTP/DELETE
```

Request Example

```
http://172.16.1.170:8888/api/v1/persons/1.4
```

Response Example

```
{
    "status": "success"
}
```

10 Working with Events

Note. The request addresses SecurOS events database.

The following requests are provided for working with the events:

- **Get Event List for Specified Camera**.

When retrieving events the additional **Event Filtering** is available.

In addition to the above operations, it is possible to send a custom event with parameters to SecurOS (see **Send Custom Event to SecurOS**).

10.1 Event Filtering

When using REST API it is possible to filter events returned in the response to the request. The filter is set in the *REST API* object settings (see **SecurOS Administration Guide**).

Selected filter will be applied when requesting *Camera* events list (see below) from the SecurOS events database.

10.2 Get Event List for Specified Camera

Resource ID

```
/api/v1/protocol/<cam_id>?start_time=<value>&stop_time=<value>
&max_count=<value>
```

Objects Access Rights

Camera — `View` (👁) and higher.

Request Parameters

- **cam_id** — identifier of the *Camera*.
- **start_time** — beginning of the sampling time period. Mandatory parameter. Is set in the `YYYYMMDD`**`T`**`HHMMSS[.SSS]` format (milliseconds are optional).
- **start_time** — end of the sampling time period. Optional parameter. Is set in the `YYYYMMDD`**`T`**`HHMMSS[.SSS]` format (milliseconds are optional).
- **max_count** — maximum number of returned records starting with the newest one. Optional parameter. If specified value is less than total records number within the specified sampling time

period, then the oldest records will be ignored.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/
api/v1/cameras/4/protocol?start_time=20200218T101000
&stop_time=20200218T101400&max_count=10
```

Response Example


```
{
  "data": {
    "actual_count": 3,
    "events": [
      {
        "action": "ARMED",
        "comment": "",
        "id": "e9f8da05-aacd-49f8-9a15-595a8602321b",
        "time": "18-02-20 10:12:12.310",
        "time_write": "18-02-20 10:12:12.963"
      },
      {
        "action": "MD_START",
        "comment": "",
        "id": "a210caa1-7829-4e1b-9486-8d5cf3d6d0b3",
        "time": "18-02-20 10:12:12.000",
        "time_write": "18-02-20 10:12:12.963"
      },
      {
        "action": "REC",
        "comment": "",
        "id": "1280b772-723d-4988-84f6-5558ced9eb75",
        "time": "18-02-20 10:12:12.548",
        "time_write": "18-02-20 10:12:12.964"
      }
    ]
  },
  "status": "success"
}
```


10.3 Send Custom Event to SecurOS

Resource ID

```
/api/v1/events/send_event?<param1_name>=<param1_value>,
[&<param2_name>=<param2_value>[& . . . ]]
```

Objects Access Rights

Not required.

Request Parameters

- **param1_name, param2_name,...** — names of the parameters, transmitted to SecurOS.
- **param1_value, param2_value,...** — values of the parameters, transmitted to SecurOS.

Note. The number of parameters is not limited. Length of the parameter names and values are not limited (in characters).

Protocol/Method

```
HTTP/POST
```

Request Example

```
http://localhost:8888/api/v1/events/send_event?plate=135
&latitude=57.6565&longitude=37.8787
```

In the example, an event with three parameters and their values is sent to SecurOS:

- `plate` — license plate number of the vehicle;
- `latitude` — latitude of the vehicle position;
- `longitude` — longitude of the vehicle position.

Response Example

```
Event (REST_API|1|RECEIVED|_body<>,_method<POST>,_path</api/v1/events/
send_event>,_peer_address<::1>,date<08-12-
20>,latitude<57.6565>,longitude<37.8787>,plate<135>,time<12:06:22.341>) has been sent
to SecurOS.
```

The following data is transmitted within the response:

- `REST_API|1|RECEIVED` — type of the object, identifier of the object and the name of the object's event, that must be intercepted and processed on the SecurOS side;
- `latitude<57.6565>,longitude<37.8787>,plate<135>` — parameters that have been transmitted within the request;
- SecurOS service data (is generated by the system automatically):
  - `_body` — text body of the request;
  - `method` — HTTP method for the request transmitting;
  - `path` — resource ID;
  - `date` — request sending date;
  - `time` — request sending time;
  - `peer_address` — IP address of the request sender. If the request has been sent from the *Computer* where the *REST API* object is created, then takes the `::1` value.

11 Working with Map

The following requests are provided for working with the *Map*:

- **Get Settings of All Maps**.
- **Get Settings of Specified Map**.

11.1 Get Settings of All Maps

Resource ID

```
/api/v1/maps
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/maps
```

Response Example


```
{
  "data": [
    {
      "id": "1",
      "map": {
        "1": {
          "color": "#ffffff",
          "image": "/api/v1/maps/1/image/1",
          "name": "Main office",
          "objects": [
            {
              "id": "1",
              "rotation": -0.66322511575784526,
              "text_elided": false,
              "text_x": 0,
              "text_y": 1,
              "type": "CAM",
              "x": 37.396694214876035,
```

```
          "y": 57.665681782497614
        },
        {
          "id": "2",
          "rotation": 2.3212879051524582,
          "text_elided": false,
          "text_x": 0,
          "text_y": 0.88774596494757085,
          "type": "CAM",
          "x": 55.785123966942152,
          "y": 37.682524729156853
        },
        {
          "id": "Dining room",
          "rotation": 0,
          "text_elided": false,
          "text_x": 0,
          "text_y": 1,
          "type": "LEVEL",
          "x": 22.933884297520663,
          "y": 82.787364935268855
        }
      ]
    },
    "2": {
      "color": "#ffffff",
      "image": "/api/v1/maps/1/image/1",
      "name": "Dining room",
      "objects": [ ]
    }
  },
  "name": "Map 1"
},
{
  "id": "2",
  "map": {
    "1": {
      "color": "#ffffff",
      "image": "/api/v1/maps/2/image/1",
      "name": "Test Lab",
      "objects": [
        {
          "id": "4",
          "rotation": 0,
          "text_elided": false,
          "text_x": 0,
          "text_y": 1,
          "type": "CAM",
          "x": 30.785123966942145,
          "y": 30.545682924392302
        },
        {
          "id": "5",
          "rotation": 0,
          "text_elided": false,
          "text_x": 0,
          "text_y": 1,
          "type": "CAM",
          "x": 64.256198347107443,
          "y": 60.234944832212854
        }
      ]
    }
```

```
      },
      "name": "Map 2"
    }
  ],
  "status": "success"
}
```


11.2 Get Settings of Specified Map

Resource ID

`/api/v1/maps/<map_id>`

Objects Access Rights

Not required.

Request Parameters

map_id — identifier of the *Map*.

Protocol/Method

`HTTP/GET`

Request Example

`http://172.16.1.170:8888/api/v1/maps/2`

Response Example


```
{
  "data": {
    "id": "2",
    "map": {
      "1": {
        "color": "#ffffff",
        "image": "/api/v1/maps/2/image/1",
        "name": "Test Lab",
        "objects": [
          {
            "id": "4",
            "rotation": 0,
            "text_elided": false,
            "text_x": 0,
            "text_y": 1,
            "type": "CAM",
            "x": 30.785123966942145,
            "y": 30.545682924392302
          },
          {
            "id": "5",
            "rotation": 0,
            "text_elided": false,
            "text_x": 0,
            "text_y": 1,
            "type": "CAM",
```

```
            "x": 64.256198347107443,
            "y": 60.234944832212854
          }
        ]
      }
    },
    "name": "Map 2"
  },
  "status": "success"
}
```

12 Working with SecurOS Auto

To send requests to SecurOS Auto it is required to **get list of Logic modules** first. For each *Logic module* configured to process HTTP requests (see **Auto User Guide** for configuration information) there are links that can be used for sending requests to SecurOS Auto or to subscribe to events from module.

The following requests are provided to work with the SecurOS Auto:

- **Get List of Logic Modules**.
- **Get Settings of All Recognizers**.
- **Get Settings of Specific Recognizer**.
- **Get List of Passages for Time Interval**.
- **Get Snapshot of Specific Passage**.
- **Managing Watchlists**.
- **Subscribe to Notifications about New Passages**.

12.1 Get List of Logic Modules

Note. Send the request to the server where the *REST API* object is located on. The port is defined by the object settings.

Resource ID

```
/api/v1/auto_logics
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/auto_logics
```

Response Example

```
{
```

```
    "data":[
       {
          "name": "LPR: Logic module 1",
          "id":"1",
          "url": "http://172.16.1.93:8899",
          "url_subscribe": "ws://172.16.1.93:8090",
          "recognizers":[
             {
                "name": "LPR: License plate recognizer 1",
                "id":"1",
                "camera": {
                      "name": "Camera 1",
                      "id": "1"
                }
             }
          ]
       }
    ],
     "status": "success"
}
```


The following parameters are returned in the response:

- **name** — name of the *Logic module* in the object tree;
- **id** — identifier of the *Logic module*;
- **url** — address to send requests to this *Logic module*;
- **url_subscribe** — address to subscribe to events of this *Logic module*;
- **recognizers** — list of *Recognizers* selected for this *Logic module*;
  - **name** — name of the *Recognizer* in the object tree;
  - **id** — identifier of the *Recognizer*;
  - **camera** — information about the camera, selected for this *Recognizer*;
    - **name** — name of the *Camera* in the object tree;
    - **id** — identifier of the *Camera*.

12.2 Get Settings of All Recognizers

Warning!

1. For the correct request execution, the **Web server** checkbox must be selected and the port number must be specified in the settings of the *Auto: Logic module*.
2. The request must be addressed to the specified port of the server where the *LPR: Logic module* object is located.

Resource ID

`/api/v1/recognizers`

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8899/api/v1/recognizers
```

Note. Response contains *Recognizers* of the *Logic module* the request is addressed to.

Response Example


```
{
"data": [
 {
"camera": {
"id": "3"
 },
“id": "1",
“name": "LPR: License plate recognizer 1"
 },
 {
"camera": {
"id": "2"
 },
"id": "2",
“name": "LPR: License plate recognizer 2"
 }
 ],
"status": "success"
}
```


The following parameters are returned in the response:

- **name** — name of the *Recognizer* in the object tree;
- **id** — identifier of the *Recognizer*;
- **camera** — information about the camera, selected for this *Recognizer*;
  - **id** — identifier of the *Camera*.

12.3 Get Settings of Specific Recognizer

Warning!

1. For the correct request execution, the **Web server** checkbox must be selected and the port number must be specified in the settings of the *Auto: Logic module*.
2. The request must be addressed to the specified port of the server where the *LPR: Logic module* object is located.

Resource ID

```
/api/v1/recognizers/<recognizer_id>
```

Objects Access Rights

Not required.

Request Parameters

recognizer_id — identifier of the *Recognizer*. The *Recognizer* must be selected in settings of *Logic module* the request is addressed to.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8899/api/v1/recognizers/1
```

Response Example

```
{
“data": {
"camera": {
"id": "3"
 },
“id": "1",
“name": "LPR: License plate recognizer 1"
 },
"status": "success"
}
```

The following parameters are returned in the response:

- **name** — name of the *Recognizer* in the object tree;
- **id** — identifier of the *Recognizer*;
- **camera** — information about the camera, selected for this *Recognizer*;
  - **id** — identifier of the *Camera*.

12.4 Get List of Passages for Time Interval

Warning!

1. For the correct request execution, the **Web server** checkbox must be selected and the port number must be specified in the settings of the *Auto: Logic module*.
2. The request must be addressed to the specified port of the server where the *LPR: Logic module* object is located.

Resource ID

```
/api/v1/recognizers/<recognizer_id>/protocol?
start_time=<value>&stop_time=<value>&max_count=<value>
```

Objects Access Rights

Not required.

Request Parameters

- **recognizer_id —** identifier of the *Recognizer*. The *Recognizer* must be selected in settings of *Logic module* the request is addressed to.
- **start_time —** beginning of the sampling time period. Mandatory parameter. Is set in the `YYYYMMDD`**`T`**`HHMMSS[.SSS]` format (milliseconds are optional).
- **stop_time —** end of the sampling time period. Optional parameter. Is set in the `YYYYMMDD`**`T`**`HHMMSS[.SSS]` format (milliseconds are optional).
- **max_count —** maximum number of returned records starting with the newest one. Optional parameter. If specified value is less than total records number within the specified sampling time period, then the oldest records will be ignored. If parameter is not defined, the response contains all records for specified period.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8899/api/v1/recognizers/1/protocol?
start_time=20200405T192103&stop_time=20200919T110603&max_count=2
```

Response Example


```
{
"data": [
 {
"camera_id": "3",
“template_country_name": "MEXICO",
"direction": "approaching",
"direction_name": "approaching",
“number": "798RBA",
"plate_rect": "0.413281,0.660417,0.505469,0.696875",
“template": "MEX_3D-3L_FED",
“time_best": "2020-10-28T15:30:10.560",
“time_enter": "2020-10-28T15:30:08.392",
“time_leave": "2020-10-28T15:30:10.560",
"track_id": "7",
“uuid": "5369c508-db7f-4361-903a-f8559c5f56db"
 },
 {
"camera_id": "3",
“template_country_name": "MEXICO",
"direction": "approaching",
"direction_name": "approaching",
“number": "314UKV",
"plate_rect": "0.727344,0.4125,0.773438,0.44375",
“template": "MEX_3D-3L_FED",
“time_best": "2020-10-28T15:30:09.395",
“time_enter": "2020-10-28T15:30:09.395",
“time_leave" "2020-10-28T15:30:09.395",
"track_id": "6",
“uuid": "b020e035-35c4-417d-8b35-e1d2ea76a3b1"
 }
 ],
```

```
"status": "success"
}
```

The following parameters are returned in the response:

- **uuid** — identifier of the passage that can be used for other passage-related requests;
- **camera_id** — identifier of the *Camera* registered the passage;
- **country** — country the license plate number belongs to;
- **direction** — vehicle movement direction;
  - `approaching` — approaching;
  - `leaving` — leaving;
  - `unknown` — failed to determine direction;
- **direction_name** — name of the direction, specified in the *Recognizer* settings;
- **number** — recognized license plate number of the vehicle;
- **plate_rect** — coordinates of the license plate in the frame (in shares);
- **template** — name of the template the recognized license plate number belongs to;
- **time_best** — timestamp of the frame when the license plate number had the best view;
- **time_enter** — time when the vehicle appears;
- **time_leave** — time when the vehicle leaves;
- **track_id** — identifier of the vehicle for inner use (not the passage identifier).

12.5 Get Snapshot of Specific Passage

Warning!

1. For the correct request execution, the **Web server** checkbox must be selected and the port number must be specified in the settings of the *Auto: Logic module*.
2. The request must be addressed to the specified port of the server where the *LPR: Logic module* object is located.

Resource ID

```
/api/v1/recognizers/<recognizer_id>/image/<uuid>
```

Objects Access Rights

Not required.

Request Parameters

- **recognizer_id** — identifier of the *Recognizer*. The *Recognizer* must be selected in settings of *Logic module* the request is addressed to.
- **uuid** — identifier of passage (can be obtained from **list of passages** or by **subscribing to notification about new events**).

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8899/api/v1/recognizers/1/image/
5369c508-db7f-4361-903a-f8559c5f56db
```

Image is sent to the REST API client in the response body and further can be saved to a file. The examples of the request headers and response body are presented below. Received image is conventionally designated as `[DATA]`.

Response Example

```
Server: Microsoft-HTTPAPI/2.0
Cache-Control: no-store, must-revalidate
Content-Type: image/jpeg
Transfer-Encoding: chunked
Date: Fri, 21 Feb 2020 07:02:07 GMT

[DATA]
```

12.6 Managing Watchlists

Warning! It is possible to work only with those watchlists that use tables of default markup.

The following requests are provided to work with the SecurOS Auto watchlists:

- **Get List of Watchlists**.
- **Set/Update Comment for License Plate Number in Watchlist**.
- **Remove License Plate Number from Watchlist**.
- **Check if License Plate Number Belongs to Watchlist**.

12.6.1 Get List of Watchlists

Warning!

1. For the correct request execution, the **Web server** checkbox must be selected and the port number must be specified in the settings of the *Auto: Logic module*.
2. The request must be addressed to the specified port of the server where the *LPR: Logic module* object is located.

Resource ID

```
/api/v1/watchlists
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

`HTTP/GET`

Request Example

```
http://172.16.1.170:8899/api/v1/watchlists
```

Warning! Deleting or adding watchlists may cause changing of identifiers of existed watchlists.

Response Example

```
{
"data": [
 {
“id": "0",
“name": "Blacklist"
 },
 {
“id": "1",
“name": "Informational list"
 },
 {
“id": "2",
“name": "Whitelist"
 }
 ],
"status": "success"
}
```

The following parameters are returned in the response:

- **id** — identifier of the watchlist;
- **name** — name of the watchlist.

12.6.2 Add License Plate Number to Watchlist or Update Its Comment

Warning!

1. For the correct request execution, the **Web server** checkbox must be selected and the port number must be specified in the settings of the *Auto: Logic module*.
2. The request must be addressed to the specified port of the server where the *LPR: Logic module* object is located.

Resource ID

`/api/v1/watchlists/<watchlist_id>/set`

Objects Access Rights

Not required.

Request Parameters

watchlist_id — identifier of watchlist (can be obtained from **list of watchlists**).

Protocol/Method

```
HTTP/POST
```

Required Header

```
Content-Type: application/json
```

Request Body

The body of the request must contain a license plate number and its comment. If the license plate number doesn't exist in the watchlist, it will be added with specified comment. If the license plate number already exists in that watchlist, its comment will be replaced with the new one.

Warning! If there were several comments for the license plate number, the request will replace all existing comments by the one specified in the request's body.

Request Body Example

```
Content-Type: application/json

{
“number": "798RBA",
"comment": "Stolen"
}
```

Request Example

```
http://172.16.1.170:8899/api/v1/watchlists/1/set
```

Response Example

```
Comment for number 798RBA has been set.
```

12.6.3 Remove License Plate Number from Watchlist

Warning!

1. For the correct request execution, the **Web server** checkbox must be selected and the port number must be specified in the settings of the *Auto: Logic module*.
2. The request must be addressed to the specified port of the server where the *LPR: Logic module* object is located.

Resource ID

```
/api/v1/watchlists/<watchlist_id>/delete
```

Objects Access Rights

Not required.

Request Parameters

watchlist_id — identifier of watchlist (can be obtained from **list of watchlists**).

Protocol/Method

```
HTTP/POST
```

Required Header

```
Content-Type: application/json
```

Request Body

The body of request must contain a license plate number to be removed from the watchlist.

Warning! All comments for specified license plate number will be removed from the watchlist.

Request Body Example

```
Content-Type: application/json

{
“number": "798RBA"
}
```

Request Example

```
http://172.16.1.170:8899/api/v1/watchlists/1/delete
```

Response Example

```
Number 798RBA has been deleted from the watchlist.
```

12.6.4 Check if License Plate Number Belongs to Watchlist

Warning!

1. For the correct request execution, the **Web server** checkbox must be selected and the port number must be specified in the settings of the *Auto: Logic module*.
2. The request must be addressed to the specified port of the server where the *LPR: Logic module* object is located.

Resource ID

```
/api/v1/watchlists/<watchlist_id>/search
```

Objects Access Rights

Not required.

Request Parameters

watchlist_id — identifier of watchlist (can be obtained from **list of watchlists**).

Protocol/Method

```
HTTP/POST
```

Required Header

```
Content-Type: application/json
```

Request Body

The request body must contain a license plate number to find.

Request Body Example

```
Content-Type: application/json

{
“number": "798RBA"
}
```

Request Example

```
http://172.16.1.170:8899/api/v1/watchlists/1/search
```

Response Example

```
That Watchlist contains number 798RBA.
```

12.7 Subscribe to Notifications about New Passages

To automatically receive information about new passages one may subscribe to them using WebSocket protocol. To do this it is required to authorize. Go through a process described for authorization in SecurOS (see **Authorization When Using WebSocket**) with some changes:

- Request the token from address defined in the `url` parameter, specified for required *Logic module* (see **Get List of Logic Modules**).
- Send the token to address defined in the `url_subscribe` parameter, specified for required *Logic module*, to start receiving passages. The subscription will be automatically created after authorization.

Pattern of a passage is the same as shown in section **Get List of Passages for Time Interval**.

13 Working with FaceX

The following requests are provided for working with the SecurOS FaceX:

- **Get FaceX Server Settings**.

13.1 Get FaceX Server Settings

Resource ID

```
/api/v1/facexconfig
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

HTTP/GET

Request Example

```
http://172.16.1.170:8888/api/v1/facexconfig
```

Response Example


```
{
  "data": [
    {
      "acs_verification_modes": {
        "multi_factor_mode": "disabled",
        "single_factor_mode": "disabled"
      },
      "facex_feed_ids": [ "1", "2" ],
      "facex_server_port": "21093",
      "server_dns": "VIDEO_SERVER_II",
      "server_ip": ""
    }
  ],
  "status": "success"
}
```


The following parameters are returned in the response:

- **server_dns** — DNS name of the *Video Server* where FaceX server is deployed;
- **server_ip** — IP address of the *Video Server* where FaceX server is deployed. This parameter is

displayed if it is specified in the correspondent *Computer* object settings;

- **facex_server_port** — FaceX server port number for interacting with the client application, specified in the *FaceX: Server* object settings;
- **facex_feed_ids** — identifiers of the *Cameras* selected to work with the FaceX server;
- **multi_factor_mode**, **single_factor_mode** — selected mode of the authentication:
  - `enabled` — mode is on;
  - `disabled` — mode id off.

14 Working with ACS

The following requests are provided to work with the ACS:

- **Get List of Objects of Specific Type**.

14.1 Get List of ACS Objects of Specific Type

Resource ID

```
/api/v1/acsconfig?type=<ACS_TYPE>&ids=<ACS_IDs>
```

Objects Access Rights

Not required.

Request Parameters

- **type** — type of ACS object, the list of which is required to obtain.

  Warning! Object type must be entered in uppercase.

- **ids** — identifiers of objects (comma-separated). Optional parameter. If not specified, the response will contain information about all objects of specified type.

Protocol/Method

HTTP/GET

Request Example

```
http://172.16.1.170:8888/api/v1/acsconfig?type=AMAG_DOOR&ids=1,2
```

For objects with linked *Cameras* the identifiers of those *Cameras* will be provided.

Response Example


```
{
  "data": [
    {
      "cameras": "2,1,3",
      “id”:"2",
      "name": "Door 2"
    },
    {
      "cameras": "4,5",
      “id”:"1",
      "name": "Door 1"
    }
```

```
  ],
  "status": "success"
}
```


The following parameters are returned in the response:

- **cameras** — list of *Cameras* associated with the object (comma-separated);
- **id** — identifier of the object;
- **name** — name of the object in the object tree.

15 Working with Macros

The following requests are provided for working with the *Macros*:

- **Get Settings of All Macros**.
- **Get Settings of Specified Macro**.
- **Run Macro**.

15.1 Get Settings of All Macros

Resource ID

```
/api/v1/macros
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/macros
```

Response Example


```
{
  "data": [
    {
      "id": "1.1",
      "local": "1",
      "name": "Macro 1"
    },
    {
      "id": "1.2",
      "local": "0",
      "name": "Macro 2"
    }
  ],
  "status": "success"
}
```

The list of returned parameters also contains the *Macro* locality flag:

- **local** — is the *Macro* local or global:
  - `1` — macro is local (may be executed only on the specified *Computer*);
  - `0` — macro is global (may be executed on all *Computer* within the SecurOS network).

15.2 Get Settings of Specified Macro

Resource ID

```
/api/v1/macros/<macro_id>
```

Objects Access Rights

Not required.

Request Parameters

macro_id — identifier of the *Macro*.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/macros/1.2
```

Response Example

```
{
  "data": {
    "id": "1.2",
    "local": "0",
    "name": "Macro 2"
  },
  "status": "success"
}
```

The list of returned parameters also contains the *Macro* locality flag:

- **local** — is the *Macro* local or global:
  - `1` — macro is local (executed only on the specified *Computer*);
  - `0` — macro is global (executed on all *Computer* within the SecurOS network).

15.3 Run Macro

Resource ID

```
/api/v1/macros/<macro_id>/run
```

Objects Access Rights

Macro — `Control` () and higher.

Request Parameters

macro_id — identifier of the *Macro*.

Protocol/Method

`HTTP/POST`

Request Example

```
http://172.16.1.170:8888/api/v1/macros/1.2/run
```

Response Example

```
{
   "status": "success"
}
```

16 Working with Computers

The following requests are provided for working with the *Computers*:

- **Get Settings of All Computers**.
- **Get Settings of Specified Computer**.

16.1 Get Settings of All Computers

Resource ID

`/api/v1/servers`

Objects Access Rights

Computer — `View` (👁) and higher.

Request Parameters

None.

Protocol/Method

`HTTP/GET`

Request Example

`http://172.16.1.170:8888/api/v1/servers`

Response Example


```
{
  "data": [
    {
      "FOLDER.audio_mode.count": "0",
      "FOLDER.login.count": "0",
      "FOLDER.network_path.count": "0",
      "FOLDER.pass.count": "0",
      "FOLDER.video_mode.count": "0",
      "SUBSCRIBE.mode.count": "0",
      "SUBSCRIBE.object.count": "0",
      "acs_database": "",
      "bookmarks_alarms_lifetime": "",
      "disable_protocol": "0",
      "display_id": "",
      "drives": "",
      "drives_a": "",
      "drives_a_r": "",
      "drives_and_folders": "",
```

```
      "drives_r": "",
      "event_filter_id": "",
      "has_own_subs": "",
      "id": "OPERATOR",
      "init_user_name": "",
      "ip_address": "",
      "is_cs": false,
      "is_profile": "1",
      "max_proto_db_size": "",
      "name": "Operator Workstation",
      "parent_id": "1",
      "record_priority": "0",
      "start_display_mode": "1",
      "state": "DISCONNECTED",
      "type": "2",
      "use_virtual_ip": "",
      "user_id": "",
      "video_database": "",
      "write_protocol_params": ""
    },
    {
      "FOLDER.audio_mode.count": "0",
      "FOLDER.login.count": "0",
      "FOLDER.network_path.count": "0",
      "FOLDER.pass.count": "0",
      "FOLDER.video_mode.count": "0",
      "SUBSCRIBE.mode.count": "0",
      "SUBSCRIBE.object.count": "0",
      "acs_database": "1",
      "bookmarks_alarms_lifetime": "0",
      "connected": true,
      "disable_protocol": "0",
      "display_id": "",
      "drives": "",
      "drives_a": "",
      "drives_a_r": "",
      "drives_and_folders": "{\"drives\":[{\"min_free_space\":0.1,\"path\":\"D:\\\\\
\",\"permissions\":{\"audio\":\"rw\",\"video\":\"rw\"}}],\"iscsi_drives\":
[],\"network_folders\":[{\"login\":\"\",\"min_free_space\":0.1,\"password
\":\"EJABDJPACKPDAFNH\",\"path\":\"\\\\\\\\storage\\\\archive\\\\\",\"permissions\":
{\"audio\":\"unused\",\"video\":\"rw\"}}]}",
      "drives_r": "",
      "event_filter_id": "-1",
      "has_own_subs": "",
      "id": "VIDEO_SERVER_I",
      "init_user_name": "",
      "ip_address": "",
      "is_cs": true,
      "is_profile": "0",
      "max_proto_db_size": "0",
      "name": "Computer VIDEO_SERVER_I",
      "parent_id": "1",
      "record_priority": "0",
      "start_display_mode": "1",
      "state": "CONNECTED",
      "type": "16",
      "use_virtual_ip": "",
      "user_id": "",
      "video_database": "",
      "write_protocol_params": "0"
    },
    {
      "FOLDER.audio_mode.count": "0",
```

```
        "FOLDER.login.count": "0",
        "FOLDER.network_path.count": "0",
        "FOLDER.pass.count": "0",
        "FOLDER.video_mode.count": "0",
        "SUBSCRIBE.mode.count": "0",
        "SUBSCRIBE.object.count": "0",
        "acs_database": "",
        "bookmarks_alarms_lifetime": "0",
        "disable_protocol": "0",
        "display_id": "",
        "drives": "",
        "drives_a": "",
        "drives_a_r": "",
        "drives_and_folders": "{\"drives\":[],\"iscsi_drives\":[],\"network_folders\":
[]}",
        "drives_r": "",
        "event_filter_id": "-1",
        "has_own_subs": "",
        "id": "OPERATOR_II",
        "init_user_name": "",
        "ip_address": "",
        "is_cs": false,
        "is_profile": "0",
        "max_proto_db_size": "0",
        "name": "Computer OPERATOR_II",
        "parent_id": "1",
        "record_priority": "0",
        "start_display_mode": "1",
        "state": "DISCONNECTED",
        "type": "2",
        "use_virtual_ip": "",
        "user_id": "",
        "video_database": "",
        "write_protocol_params": "0"
      }
    ],
    "status": "success"
}
```


List of the returned parameters of the *Computer* also contains its role and status:

- **is_cs** — is this *Computer* a *Configuration Server* or not:
  - `true` — yes;
  - `false` — no.
- **connected** — current state of the computer:
  - `true` — connected;
  - `false` — disconnected.

16.2 Get Settings of Specified Computer

Resource ID

`/api/v1/servers/<computer_DNS_name>`

Objects Access Rights

Computer — `View` (👁) and higher.

Request Parameters

computer_DNS_name — DNS name of the *Computer*.

Protocol/Method

`HTTP/GET`

Request Example

`http://172.16.1.170:8888/api/v1/servers/VIDEO_SERVER_II`

Response Example


```
{
  "data": {
    "FOLDER.audio_mode.count": "0",
    "FOLDER.login.count": "0",
    "FOLDER.network_path.count": "0",
    "FOLDER.pass.count": "0",
    "FOLDER.video_mode.count": "0",
    "SUBSCRIBE.mode.count": "0",
    "SUBSCRIBE.object.count": "0",
    "acs_database": "",
    "bookmarks_alarms_lifetime": "0",
    "connected": false,
    "disable_protocol": "0",
    "display_id": "",
    "drives": "",
    "drives_a": "",
    "drives_a_r": "",
    "drives_and_folders": "{\"drives\":[],\"iscsi_drives\":[],\"network_folders\":[]}",
    "drives_r": "",
    "event_filter_id": "-1",
    "has_own_subs": "",
    "id": "VIDEO_SERVER_II",
    "init_user_name": "",
    "ip_address": "172.16.12.174",
    "is_cs": false,
    "is_profile": "0",
    "max_proto_db_size": "0",
    "name": "Computer VIDEO_SERVER_II",
    "parent_id": "1",
    "record_priority": "0",
    "start_display_mode": "1",
    "state": "DISCONNECTED",
    "type": "16",
    "use_virtual_ip": "",
    "user_id": "",
    "video_database": "",
    "write_protocol_params": "0"
  },
  "status": "success"
}
```


List of the returned parameters of the *Computer* also contains it role and status:

- **is_cs** — is this *Computer* a *Configuration Server* or not:

  - `true` — yes;
  - `false` — no.
- **connected** — current state of the computer:
  - `true` — connected;
  - `false` — disconnected.

17 Get License Information

Resource ID

`/api/v1/licenseinfo`

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

`HTTP/GET`

Request Example

`http://172.16.1.170:8888/api/v1/licenseinfo`

Warning! For unlimited key positions the 99999999 value will be displayed.

Response Example


```
{
    "licenseInfo": {
        "userAccounts": {
            "licenseQuantity": 999999999,
            "utilizedQuantity": 3
        },
        "videoServers": {
            "licenseQuantity": 5,
            "utilizedQuantity": 4
        },
        "cameras": {
            "licenseQuantity": 100,
            "utilizedQuantity": 50
        },
        "microphones": {
            "licenseQuantity": 10,
            "utilizedQuantity": 2
        },
        "relaysAndSensors": {
            "licenseQuantity": 10,
            "utilizedQuantity": 10
        },
        "LPRdetectorsLite": {
            "licenseQuantity": 3,
            "utilizedQuantity": 1
```

```
            },
            "LPRdetectors": {
                "licenseQuantity": 3,
                "utilizedQuantity": 1
            },
            "FaceXdetectors": {
                "licenseQuantity": 1,
                "utilizedQuantity": 1
            }
        },
        "status": "success"
    }
```


The following parameters are returned in the response:

- **userAccounts** — quantity of *User* objects;
- **cameras** — quantity of *Camera* objects;
- **microphones** — quantity of *Microphone* objects;
- **relayAndSensors** — total quantity of *Sensor* and *Relay* objects;
- **LPRdetectorsLite** — quantity of *LPR: License Plate Recognizer 12 fps* objects;
- **LPRdetectors** — quantity of *LPR: License Plate Recognizer* objects;
- **FaceXdetectors** — quantity of FaceX recognition channels (*Cameras* selected in *FaceX: Server* objects settings);

For each parameter there are following value listed:

- **licenseQuantity** — quantity of licenses defined in the license key;
- **utilizedQuantity** — quantity of used licenses.

18 Working with Subscriptions

One can work with subscriptions via HTTP or WebSocket protocols:

- **Working via HTTP**.
- **Working via WebSocket**.

18.1 Working via HTTP

The following requests are provided for working with event subscriptions via HTTP:

- **Subscribe to event (HTTP)**.
- **Get Information about All Active Subscriptions (HTTP)**.
- **Get Information about Specified Subscription (HTTP)**.
- **Update Subscription (HTTP)**.
- **Delete Subscription (HTTP)**.

18.1.1 Subscribe to event (HTTP)

Resource ID

```
/api/v1/events/subscriptions
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

```
HTTP/POST
```

Required Header

```
Content-Type: application/json
```

Request Body

To subscribe to the specified events of the specified objects the subscription filter must be passed in the request body. If the filter is created with no parameters then client will be subscribed to all events of all SecurOS objects.

Request Body Example

```
{
  "callback": "http://client_app_address",
  "filter": {
       "action":"ALARM",
       "id": "4",
       "type": "CAM"
       }
}
```

The following parameters are passed in the request body:

- **callback** — address of the service where notifications about events will be sent to. Mandatory parameter;
- **filter.action** — type of the event for which the subscription is created. Optional parameter. If not set the notifications about all events of the specified object(s) will be sent.
- **filter.id** — identifier of the object for which the subscription is created. Optional parameter. If not set the notifications about all events of all objects of the specified type will be sent.
- **filter.type** — type of the object for events of which the subscription is created. Optional parameter. If not set the notifications about all events of all SecurOS objects will be sent.

Request Example

```
http://172.16.1.170:8888/api/v1/events/subscriptions
```

Response Example

```
{
  "data": {
    "callback": "http://client_app_address",
    "filter": {
      "action": "ALARM",
      "id": "4",
      "type": "CAM"
    },
    "id": "1f2e91b7-9a97-4838-b39a-f424c1a04c57"
  },
  "status": "success"
}
```

Response returns the identifier of the subscription.

18.1.2 Get Information about All Active Subscriptions (HTTP)

Resource ID

```
/api/v1/events/subscriptions
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/events/subscriptions
```

Response Example


```
{
  "data": {
    "callback": "http://client_app_address",
    "filter": {
      "action": "START_REC",
      "id": "3",
      "type": "CAM"
    },
    "id": "9d4b2852-36c4-4476-9992-af87107a3a02"
  },
  "status": "success"
} {
  "data": [
    {
      "callback": "http://client_app_address",
      "filter": {
        "action": "ALARM",
        "id": "4",
        "type": "CAM"
      },
      "id": "1f2e91b7-9a97-4838-b39a-f424c1a04c57"
    },
    {
      "callback": "http://client_app_address",
      "filter": {
        "action": "ARM",
        "id": "3",
        "type": "CAM"
      },
      "id": "d7c6c84d-c2d9-4cee-958b-9187335f36ea"
    }
  ],
  "status": "success"
}
```


The following parameters are returned in the response:

- **callback** — address of the service where notifications about events will be sent to;
- **filter.action** — type of the event for which there is a subscription;
- **filter.id** — identifier of the object for which there is a subscription;
- **filter.type** — type of the object for events of which there is a subscription;
- **subscription_id** — identifier of the subscription.

18.1.3 Get Information about Specified Subscription (HTTP)

Resource ID

```
/api/v1/events/subscriptions/<subscription_id>
```

Objects Access Rights

Not required.

Request Parameters

subscription_id — identifier of the subscription.

Protocol/Method

```
HTTP/GET
```

Request Example

```
http://172.16.1.170:8888/api/v1/events/subscriptions/
8382fe09-7312-438b-8350-1884b0dc68bc
```

Response Example

```
{
  “data": {
    "callback": "http://localhost",
    "filter": {
      "action": "ALARM",
      “id":"4",
      "type": "CAM"
    },
    "id": "8382fe09-7312-438b-8350-1884b0dc68bc"
  },
  "status": "success"
}
```

The following parameters are returned in the response:

- **callback** — address of the service where notifications about events will be sent to;
- **filter.action** — type of the event for which there is a subscription;
- **filter.id** — identifier of the object for which there is a subscription;
- **filter.type** — type of the object for events of which there is a subscription;
- **subscription_id** — identifier of the subscription.

18.1.4 Update Subscription (HTTP)

Resource ID

```
/api/v1/events/subscriptions/<subscription_id>
```

Request Parameters

subscription_id — identifier of the subscription.

Protocol/Method

```
HTTP/PUT
```

Required Header

```
Content-Type: application/json
```

Request Body

The new parameters of the subscription filter must be passed in the request body.

Request Body Example

```
Content-Type: application/json

{
  "callback": "http://client_app_address",
  "filter": {
       "action":"ALARM",
       "id": "4",
       "type": "CAM"
       }
}
```

Request Example

```
http://172.16.1.170:8888/api/v1/events/subscriptions/
d7c6c84d-c2d9-4cee-958b-9187335f36ea
```

Response Example

```
{
  "data": {
    "callback": "http://client_app_address",
    "filter": {
      "action": "ALARM",
      "id": "4",
      "type": "CAM"
    },
    "id": "1f2e91b7-9a97-4838-b39a-f424c1a04c57"
  },
  "status": "success"
}
```

18.1.5 Delete Subscription (HTTP)

Resource ID

```
/api/v1/events/subscriptions/<subscription_id>
```

Objects Access Rights

Not required.

Request Parameters

subscription_id — identifier of the subscription.

Protocol/Method

```
HTTP/DELETE
```

Request Example

```
http://172.16.1.170:8888//api/v1/events/subscriptions/
1f2e91b7-9a97-4838-b39a-f424c1a04c57
```

Response Example

```
{
  “data": {
    "callback": "http://client_app_address",
    "filter": {
      "action": "ALARM",
      “id":"4",
      "type": "CAM"
    },
    "id": "1f2e91b7-9a97-4838-b39a-f424c1a04c57"
  },
  "status": "success"
}
```

18.2 Working via WebSocket

Warning! The authorization is required before executing requests (see **Authorization When Using WebSocket**).

The following requests are provided for working with subscriptions via WebSocket:

- **Common Request**.
- **Subscribe to Event (WebSocket)**.
- **Subscribe to State Change (WebSocket)**.
- **Delete All Subscriptions (WebSocket)**.
- **Delete Subscription (WebSocket)**.
- **Update Subscription (WebSocket)**.

Note. WebSocket Extensions and WebSocket Subprotocols are not supported.

18.2.1 Monitoring SecurOS server connection state

To check current state of the WebSocket connection with SecurOS server, the current server time is requested.

Request Example

```
{
    "type":"get_server_time"
}
```

Request Parameters

- **type** — request type (requesting current SecurOS server time). Mandatory parameter.

Response Example

```
{
  "serverTime": "We jun 17 16:57:11 2020"
}
```

18.2.2 Common Request

When using the WebSocket protocol, all operations with subscriptions (subscribe to an event / change of state and delete a subscription) can be performed in one request (see the example below). Examples of separate operations with the subscriptions are discussed in the relevant sections.

Request Example

```
{
    “type":"subscribe",
    “id":12345,
    “data":{
        “clear":false,
        “add_rules":[
            {
                “type":"CAM",
                “id":"1",
                “states":[
                    “attached",
                    “armed",
                    “alarmed"
                ],
                “action":"STATE_CHANGED"
            },
            {
                “type":"LPR_CAM",
                “id":"1",
                “events":[
                    “CAR_LP_RECOGNIZED"
                ],
                “action":"EVENT"
            }
        ],
        “delete_rules":[
            {
                “type":"CAM",
                “id":"2",
                “action":"STATE_CHANGED"
            }
```

```
            ]
        }
    }
```

Request Parameters

- **type** — type of the request (subscription control). Mandatory parameter.
- **id** — numeric message identifier. An optional parameter that is sent by the server in response to an incorrect request.
- **data.clear** — optional parameter. Delete all subscriptions. Possible values:
  - `false` — do not delete current subscriptions (default value);
  - `true` — delete all current subscriptions.
- **data.add_rules** — list of the rules that will be added to the subscription:
  - **data.add_rules[n].type** — type of the object(-s) to subscribe to;
  - **data.add_rules[n].id** — `id` of the object to subscribe to. If the parameter is not specified, then subscribe to all objects of a given type;
  - **data.add_rules[n].states** — list of the state type names for subscribing to states (`STATE_CHANGED`), the changes of which will be sent to the client (see **Object State Change Message**). If the parameter is omitted then the client will receive all types. One can subscribe to change of the following states only:
    - **data.states.alarmed** — alarm is not detected/alarm detected;
    - **data.states.armed** — object is armed/disarmed;
    - **data.states.attached** — object is enabled/disabled.
  - **data.add_rules[n].events** — list of the event names for the event subscription (`EVENT`). If the parameter is omitted then the client will receive all events;
  - **data.add_rules[n].action** — type of the subscription. Two values are valid in the current version of the REST API:
    - `STATE_CHANGED` — notification about changing state;
    - `EVENT` — receiving events.
- **data.delete_rules** — list of the rules to be deleted from the subscription:
  - **data.add_rules[n].type** — type of the object(-s) that will be deleted from the subscription;
  - **data.add_rules[n].id** — `id` of the object that will be deleted from the subscription. If there is no such object, an error is returned. If this parameter is not specified then the subscription to all objects of the given type is canceled regardless of how the subscription has been made (for example, subscription to the object or to the objects type). If the `id` of an existing object is specified, but there was a subscription by type before that, then conversion to a subscription by `id` is performed for all objects except the excluded one;
  - **data.delete_rules[n].action** — textual identifier of the event.

Response

If the request does not contain errors, then there is no response to the command. For all `STATE_CHANGED` rules server sends states of the objects.

18.2.3 Subscribe to Event (WebSocket)

To subscribe to an event, including ACS objects events, you need to specify parameters of the action having `EVENT` type in the `add_rules` section.

Request Example

```
{
  "type": "subscribe",
  "id": 12345,
  "data": {
    "add_rules": [
      {
        "type": "LPR_CAM",
        "id": "1",
        "events": [
          "CAR_LP_RECOGNIZED"
        ],
        "action": "EVENT"
      }
    ]
  }
}
```

Request Parameters

- **type** — type of the request (subscription control). Mandatory parameter.
- **id** — numeric message identifier. An optional parameter that is sent by the server in response to an incorrect request.
- **data.add_rules** — list of the rules that will be added to the subscription:
  - **data.add_rules[n].type** — type of the object(-s) to subscribe to;
  - **data.add_rules[n].id** — `id` of the object to subscribe to. If the parameter is not specified, then subscribe to all objects of a given type;
  - **data.add_rules[n].events** — list of the event names for the event subscription (`EVENT`). If the parameter is omitted then the client will receive all events;
  - **data.add_rules[n].action** — type of the subscription (`EVENT`).

Response

If the request does not contain errors, then there is no response to the command.

18.2.4 Subscribe to State Change (WebSocket)

To subscribe to a state change you need to specify parameters of the action having `STATE_CHANGED` type in the `add_rules` section.

Request Example

```
{
  "type": "subscribe",
  "id": 12345,
  "data": {
    "add_rules": [
      {
        "type": "CAM",
        "id": "1",
        "states": [
          "attached",
          "armed",
          "alarmed"
        ],
        "action": "STATE_CHANGED"
      }
```

```
    ]
  }
}
```

Request Parameters

- **type** — type of the request (subscription control). Mandatory parameter.
- **id** — numeric message identifier. An optional parameter that is sent by the server in response to an incorrect request.
- **data.add_rules** — list of the rules that will be added to the subscription:
  - **data.add_rules[n].type** — type of the object(-s) to subscribe to;
  - **data.add_rules[n].id** — `id` of the object to subscribe to. If the parameter is not specified, then subscribe to all objects of a given type;
  - **data.add_rules[n].states** — list of the state types names for subscribing to states (`STATE_CHANGED`), the change of which will be sent to the client. If the parameter is omitted set client will receive all types. Names of all state types are represented below (see **Object State Change Message**). One can subscribe to change of the following states only:
    - **data.states.alarmed** — alarm is not detected/alarm detected;
    - **data.states.armed** — object is armed/disarmed;
    - **data.states.attached** — object is enabled/disabled.
  - **data.add_rules[n].events** — list of the event names for the event subscription (`EVENT`). If the parameter is omitted then the client will receive all events;
  - **data.add_rules[n].action** — type of the subscription (`STATE_CHANGED`).

Response

If the request does not contain errors, then there is no response to the command. For all `STATE_CHANGED` rules server sends states of the objects.

18.2.5 Get Information about All Active Subscriptions (WebSocket)

List of active subscriptions issued via WebSocket protocol, may be obtained with the help of the HTTP GET request.

> **Warning!** The request can be executed locally only on the computer on which the REST API object is created and configured.

Resource ID

```
/api/v1/events/subscriptions/ws
```

Objects Access Rights

Not required.

Request Parameters

None.

Protocol/Method

HTTP/GET

Request Example

```
http://localhost:8888/api/v1/events/subscriptions/ws
```

Response Example


```
{
  "data": [
    {
      "clientHost": "localhost",
      "subscribesByObjTypes": [
        {
          "objectType": "CAM",
          "subscribesByActions": [ ],
          "subscribesByObjIds": [
            {
              "objectId": "1",
              "subscribesByActions": [
                {
                  "actionType": "Changed",
                  "states": [ "attached", "armed", "alarmed" ]
                }
              ]
            },
            {
              "objectId": "2",
              "subscribesByActions": [
                {
                  "actionType": "Event",
                  "events": [ ]
                }
              ]
            }
          ]
        }
      ]
    },
    {
      "clientHost": "localhost",
      "subscribesByObjTypes": [
        {
          "objectType": "CAM",
          "subscribesByActions": [ ],
          "subscribesByObjIds": [
            {
              "objectId": "1",
              "subscribesByActions": [
                {
                  "actionType": "Changed",
                  "states": [ "attached", "armed", "alarmed" ]
                }
              ]
            },
            {
              "objectId": "2",
              "subscribesByActions": [
                {
                  "actionType": "Event",
```

```
            "events": [ ]
          }
        ]
      }
    ]
  }
]
},
{
  "clientHost": "localhost",
  "subscribesByObjTypes": [
    {
      "objectType": "CAM",
      "subscribesByActions": [ ],
      "subscribesByObjIds": [
        {
          "objectId": "1",
          "subscribesByActions": [
            {
              "actionType": "Changed",
              "states": [ "attached", "armed", "alarmed" ]
            }
          ]
        },
        {
          "objectId": "2",
          "subscribesByActions": [
            {
              "actionType": "Event",
              "events": [ ]
            }
          ]
        }
      ]
    }
  ]
},
{
  "clientHost": "::ffff:127.0.0.1",
  "subscribesByObjTypes": [
    {
      "objectType": "CAM",
      "subscribesByActions": [ ],
      "subscribesByObjIds": [
        {
          "objectId": "1",
          "subscribesByActions": [
            {
              "actionType": "Changed",
              "states": [ "attached", "armed", "alarmed" ]
            }
          ]
        },
        {
          "objectId": "2",
          "subscribesByActions": [
            {
              "actionType": "Event",
              "events": [ ]
            }
          ]
        }
      ]
```

```
        }
      ]
    }
  ],
  "status": "success"
}
```

18.2.6 Delete All Subscriptions (WebSocket)

To delete all subscription it enough to set the **data.clear** parameter to `true` (see below).

Request Example

```
{
  "type": "subscribe",
  "id": 12345,
  "data": {
    "clear": true
  }
}
```

18.2.7 Delete Subscription (WebSocket)

To delete a subscription, you need to specify action parameters with the `EVENT` type (delete event subscription) or `STATE_CHANGED` type (delete state change subscription) in the `delete_rules` section.

Request Example

```
{
  “type": "subscribe",
  “id": 12345,
  “data": {
    “delete_rules": [
      {
        “type": "CAM",
        “id": "2",
        “action": "STATE_CHANGED"
      }
    ]
  }
}
```

Request Parameters

- **type** — type of the request (subscription control). Mandatory parameter.
- **id** — numeric message identifier. An optional parameter that is sent by the server in response to an incorrect request.
- **data.delete_rules** — list of the rules to be deleted from the subscription:
  - **data.add_rules[n].type** — type of the object(-s) that will be deleted from the subscription;
  - **data.add_rules[n].id** — `id` of the object that will be deleted from the subscription. If there is no such object, an error is returned. If this parameter is not specified then the subscription to all objects of the given type is canceled regardless of how the subscription has been made (for example, subscription to the object or to the objects type). If the `id` of an existing object is specified, but there was a subscription by type before that, then conversion to a subscription by `id` is performed for all objects except the excluded one;
  - **data.delete_rules[n].action** — textual identifier of the event (`EVENT` or `STATE_CHANGED`).

Note. The states of the object for which a subscription was previously created is not required to be listed. Subscriptions to all possible states of the object will be deleted.

Response

If the request does not contain errors, then there is no response to the command.

18.2.8 Update Subscription (WebSocket)

Subscription update is carried out in two stages:

1. Deletion of the current subscription (see **Delete Subscription (WebSocket)**);
2. Creation of the new subscription for the same object with new parameters (see **Subscribe to Event (WebSocket)** or **Subscribe to State Change (WebSocket)**).

18.2.9 WebSocket Server Messages

When using WebSocket the following server notifications are possible:

- **Object State Change Message**.
- **Event Message**.
- **Object Deletion Message**.
- **Error Message**.

18.2.9.1 Object State Change Message

The message is sent to all clients who have a subscription to change the state of the object (`STATE_CHANGED`). Acknowledgment of a state change message is not required.

```
{
“type": "state",
“data": {
"type": "CAM",
“id":"1",
"ticks": 501586,
"time": "2017-02-02T16:10:07.241",
"states": {
"alarmed": false,
"armed": false,
"attached": true
 }
 }
}
```

Message parameters:

- **type** — type of the message `state` (state changing);
- **data.type** — type of the object whose state has been changed;
- **data.type** — identifier of the object whose state has been changed;
- **data.ticks** — millisecond counter (from the WebSocket server start). It can be used for additional control in case of delays when transmitting messages over the network;
- **data.time** — date and time of sending the message to the client (according to the server system time);

- **data.states** — state flags (`true`/`false`). The associative array contains only those flags that have been modified:
  - **data.states.alarmed** — alarm was registered;
  - **data.states.armed** — object is armed;
  - **data.states.attached** — object is connected to the network.

18.2.9.2 Event Message

The message is sent to all clients who have a subscription to event of the object (`EVENT`). Acknowledgment of an event message is not required.


```
{
  "type": "event",
  “data": {
    "type": "LPR_CAM",
    “id":"1",
    "action": "CAR_LP_RECOGNIZED",
    "ticks": 501586,
    "time": "2018-03-26T21:23:16.122",
    "parameters": {
      "best_view_date_time": "26-03-2018 21:23:15.898",
      "best_view_mask_id": "6.0",
      "camera_id": "6",
      "direction_id": "1",
      "direction_name": "approaching",
      "number": "T345 LYW",
      "number_utf8": "T345 LYW",
      "plate_bottom_i": "0.88958333333333328",
      "plate_left_i": "0.096093750000000006",
      "plate_right_i": "0.22421875000000002",
      "plate_top_i": "0.85624999999999996",
      "recognizer_id": "1",
      "recognizer_name": "License plate recognizer 1",
      "recognizer_type": "LPR_CAM",
      "speed": "0.94696974754333496",
      "template_country_id": "7",
      "template_country_iso_code": "GB",
      "template_country_name": "UNITED_KINGDOM",
      "template_name": "GB_7S_R",
      "time_enter": "26-03-2018 21:23:14.998",
      "time_leave": "26-03-2018 21:23:15.965",
      "track_id": "18",
      "tracked_out": "1",
      "units": "0",
      "velocity": "0.94696974754333496",
      "weight": "1684.4208984375"
    }
  }
}
```


Message parameters:

- **type** — type of the message `event` (event);
- **data.type** — type of the object that generated an event;
- **data.type** — identifier of the object that generated an event;
- **data.ticks** — millisecond counter (from the WebSocket server start). It can be used for additional control in case of delays when transmitting messages over the network;

- **data.time** — event generation timestamp;
- **data.parameters** — list of the event parameters (see example below).

General event parameters

Note. The example lists the main parameters for the `CAR_LP_RECOGNIZED` event of the SecurOS Auto Module.

Parameter	Description
`template_country_iso_code`	The character code of the country to which the recognized vehicle number belongs, in ISO 3166 format. Presented as the Alpha-2 universal two-digit code. For example: • `US` — USA. • `RU` — Russia. • etc.
`template_country_name`	The name of the vehicle license plate number template that was used to determine whether the recognized license plate belongs to a specific country. The decoding of the template names is given in the `C:\Program Files (x86)\ISS\SecurOS\Modules\auto\LPR\XXX` folders, where `XXX` is the name of the country whose vehicle license plate templates were used.

18.2.9.3 Object Deletion Message

Object deletion message is sent to all clients who have a subscription to an object removed from the SecurOS *Object tree*.

```
{
  "type": "event",
  "data": {
    "type": "CAM",
    "id": "1.8",
    "action": "DELETED",
    "time": "2017-04-30T14:10:09.315"
  }
}
```

Message parameters:

- **type** — type of the message `event` (event);
- **data.type** — type of the deleted object;
- **data.type** — identifier of the deleted object;
- **data.time** — date and time of sending the message to the client (according to the server system time);
- **data.action** — event code `DELETED` (object has been deleted).

18.2.9.4 Error Message

An error message is sent in response to an incorrect client request:

```
{
    "type":"error",
    "data":{
        "request_id":"12345",
        "message":"Object not found",
        "error":"OBJECT_NOT_FOUND"
    }
}
```

Message parameters:

- **type** — type of the message ("error"): error message;
- **data.request_id** — request identifier passed in the request;
- **data.message** — error text message;
- **data.error** — text error code.

Error codes are represented in the Table 1.

Table 1. Error Codes

Error code	Description
BAD_JSON	Syntax error when parsing JSON message
UNKNOWN_MESSAGE_TYPE	Unknown request type
INCORRECT_MESSAGE_STRUCTURE	Incorrect message structure (mandatory parameter is omitted, invalid field type, etc.)
UNKNOWN_OBJECT_TYPE	Unknown object type
UNKNOWN_SUBSCRIBE_ACTION	Unknown subscription code
OBJECT_NOT_FOUND	Object not found
UNKNOWN_STATE_IDENTIFIER	Invalid state name in state change subscription (`STATE_CHANGED`)

19 HTTP Status Codes

To inform user about request result the following standard HTTP status codes are used:

Table 2. State Codes

Code	Description
200 (OK)	Request successful
400 (Bad Request)	Invalid request parameters
401 (Unauthorized)	Authorization error
403 (Forbidden)	User not having the necessary permissions to the resource
404 (Not Found)	SecurOS object with specified ID not found
408 (Request Timeout)	Timeout expired
503 (Service Unavailable)	Server cannot handle the request

20 Technical Support Information

Current section contains service information that is necessary on addressing to Intelligent Security Systems Technical Support.

Note. Collected data have to be send to the Intelligent Security Systems Technical Support Team (see **Getting Technical Support**).

To ensure quick technical support, prepare the following technical information:

Warning! Data in items marked by "*" are necessary to report.

1. (*) User (customer) name to address to.
2. (*) Organization name.
3. (*) User (or organization) contacts: phone, e-mail.
4. Name of a personal Intelligent Security Systems manager (on Intelligent Security Systems authorized partner case). Otherwise, give the following data:
   - Company where the hardware and software components were purchased.
   - Actions proposed to solve the problems announced by a partner from whom the product was purchased.
5. (*) Problem description.
6. (*) Actions results in the problem.
7. List of changes which result to the problem in case of applying after some changes in system settings/configuration.
8. System and diagnostic information on computer and SecurOS system configuration obtained from the **SystemInfo** utility (see **SecurOS Administration Guide** for detailed information about utility).

   If it is impossible to run the utility provide the following information:
   - (*) Guardant keys identifiers and Dallas code;

   Note. Equipment Dallas code can be found by the **Hardware Report Utility** (see **SecurOS Administration Guide** for detailed information about utility).

   - (*) name and version of the installed Intelligent Security Systems company software.
   - total number of video servers and monitoring (operator) workstations in the system;
   - operating system (name and service pack version).
9. Another useful information, if possible. For example:
   - computer equipment configuration.
   - central processors load.
   - main and virtual memory used volumes.

- network load.
- network and network neighborhood configuration.